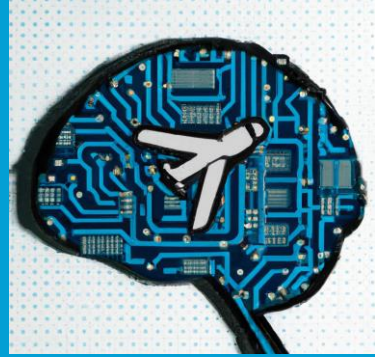
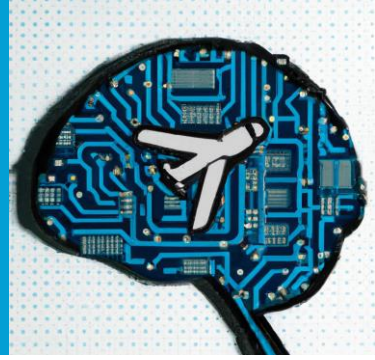




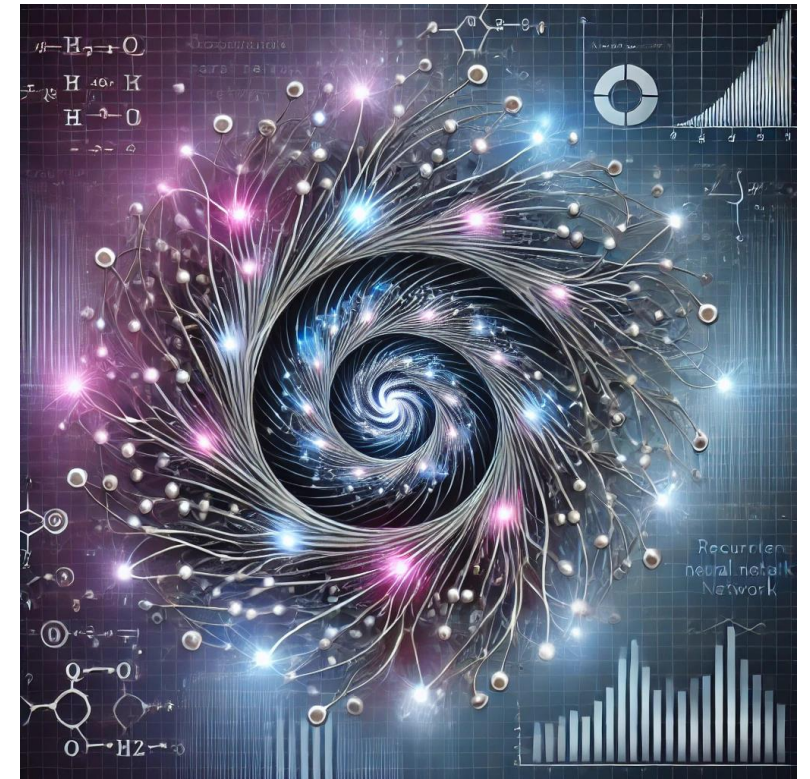
Recurrent neural networks

Lecture objectives

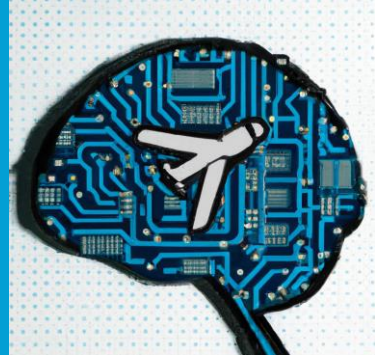


Students can:

- List different types of neural networks for processing input sequences and describe their properties
- Explain the challenges involved in training recurrent neural networks, and the strategies devised to tackle approach these challenges
- Apply RNNs to sequential tasks



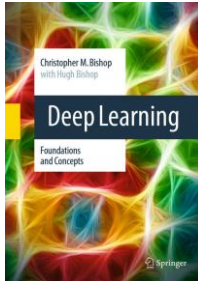
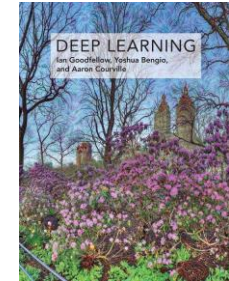
Recommended reading



“Deep learning”, by Ian GoodFellow, Yoshua Bengio, and Aaron Courville
Chapter 10: Sequence modeling: recurrent and recursive nets.

“Deep learning: Foundations and concepts”, by Bishop, C. M., & Bishop, H. (2023).
Available via TU Delft [here](#).

Chapter 12: Transformers.



Scientific articles referenced in the slides.

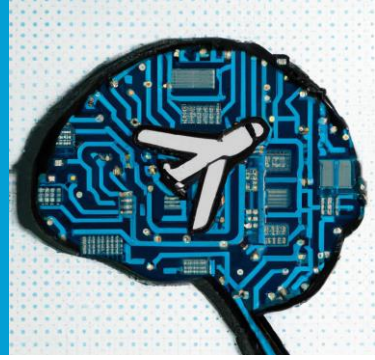
Interactive reader on github:

https://github.com/guidoAI/recurrent_networks

https://mybinder.org/v2/gh/guidoAI/recurrent_networks/HEAD?urlpath=%2Fdoc%2Ftree%2FRecurrent+neural+networks.ipynb

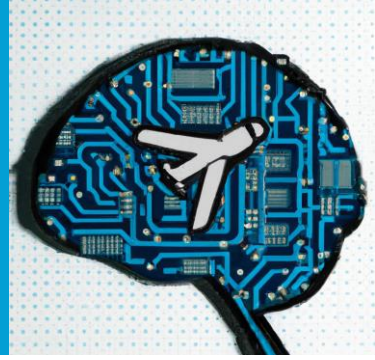


Sequential data



Many real-world tasks involve sequential data

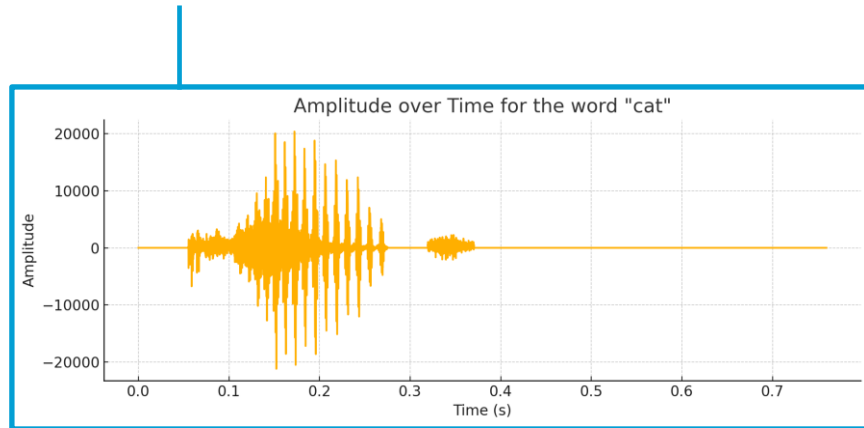
Sequential data



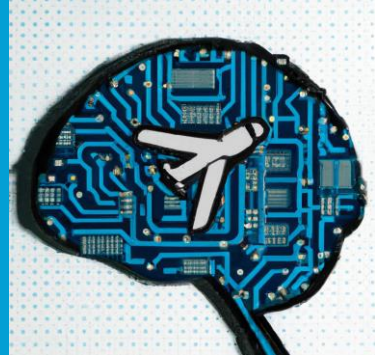
Many real-world tasks involve sequential data

Language:

... The cat sleeps on the couch. ...



Sequential data



Many real-world tasks involve sequential data

The stock market:

Market Summary > AEX index

905,63

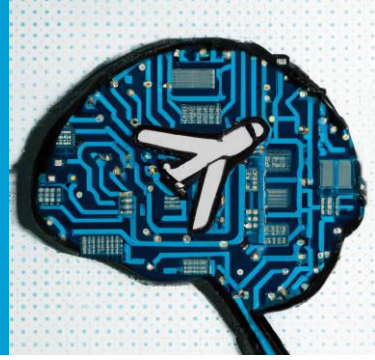
+146.35 (19.27%) ↑ past year

15 Aug, 17:30 CEST • Disclaimer

1D 5D 1M 6M YTD 1Y 5Y Max



Sequential data

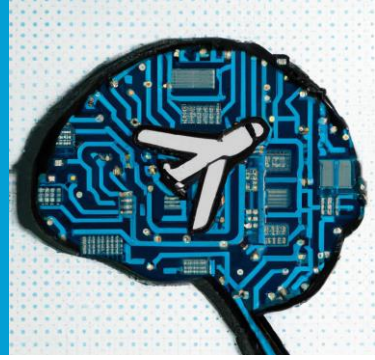


Many real-world tasks involve sequential data

Robot sensory and control signals:

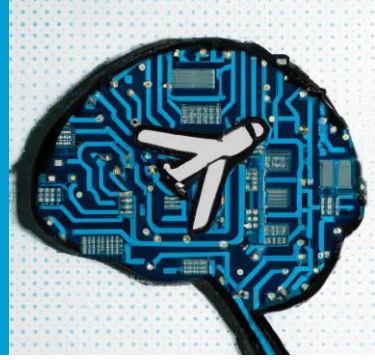


Sequential data



How to process sequential data with a neural network?

Sequential data



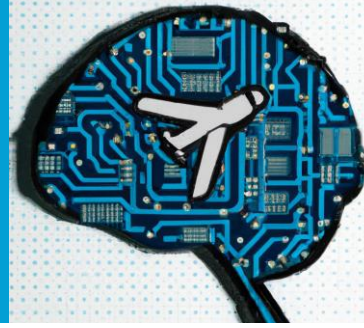
How to process sequential data with a neural network?

Input an input sequence directly into the neural network:

$$\mathbf{o}_t = N(\mathbf{x}_{1:t})$$

Where N is a feedforward neural network, and $\mathbf{x}_{1:t}$ the sequence of inputs.

Sequential data



How to process sequential data with a neural network?

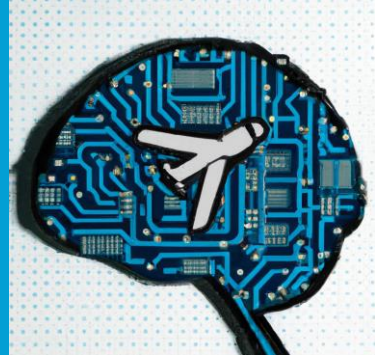
Input an input sequence directly into the neural network:

$$\mathbf{o}_t = N(\mathbf{x}_{1:t})$$

Where N is a feedforward neural network, and $\mathbf{x}_{1:t}$ the sequence of inputs.

✗ Long sequences & high input dimensions will lead to many parameters

Sequential data

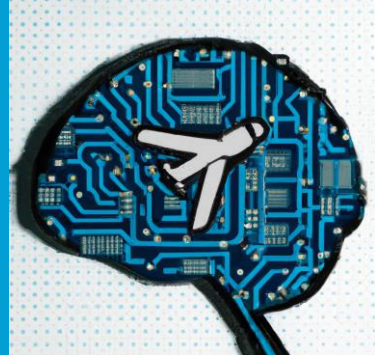


How to process sequential data with a neural network?

Input a time window T from the input sequence into the neural network:

$$\mathbf{o}_t = N(\mathbf{x}_{t-T:t})$$

Sequential data

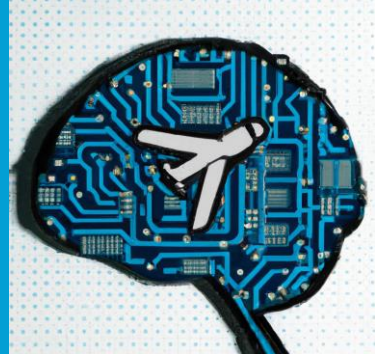


How to process sequential data with a neural network?

Input a time window T from the input sequence into the neural network:

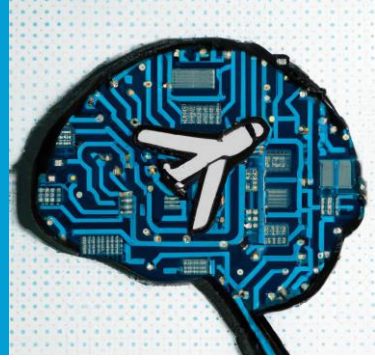
$$\mathbf{o}_t = N(\mathbf{x}_{t-T:t})$$

✗ Network can only learn short-term input patterns



Transformers

Transformers

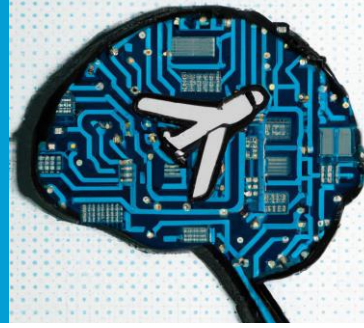


One of the main advances in AI in the past decade is the introduction of **transformers**, which were invented for language processing.



Vaswani, A. et al. (2017). Attention is all you need. *arXiv preprint arXiv:1706.03762*.

Transformers

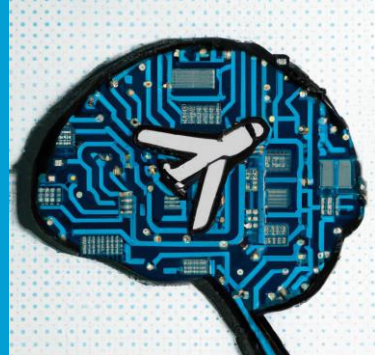


One of the main advances in AI in the past decade is the introduction of **transformers**, which were invented for language processing.

For input vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t$ it produces output vectors $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_t$.



Transformers



One of the main advances in AI in the past decade is the introduction of **transformers**, which were invented for language processing.

For input vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t$ it produces output vectors $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_t$.

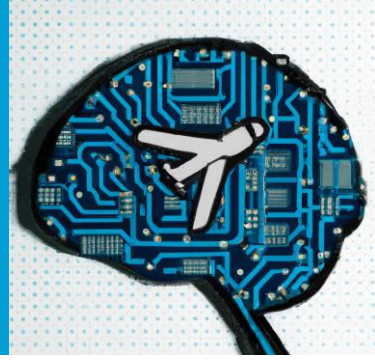
Words are transformed into vectors via a learned “embedding”:

The cat walks $\longrightarrow \mathbf{x}_{\text{the}}, \mathbf{x}_{\text{cat}}, \mathbf{x}_{\text{walks}}$



Vaswani, A. et al. (2017). Attention is all you need. *arXiv preprint arXiv:1706.03762*.

Transformers



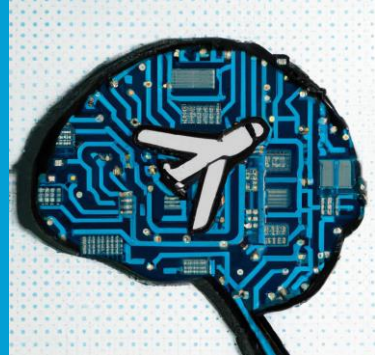
One of the main advances in AI in the past decade is the introduction of **transformers**, which were invented for language processing.

For input vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t$ it produces output vectors $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_t$.

It can process sequences of varying length.



Transformers



The core new element in transformers is a process of “self-attention”.

[home](#) / [blog](#) /

TRANSFORMERS FROM SCRATCH

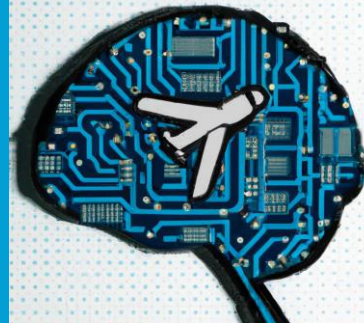
18 Aug 2019 · [code on github](#) · [video lecture](#)

Transformers are a very exciting family of machine learning architectures. Many good tutorials exist (e.g. [1, 2]) but in the last few years, transformers have mostly become simpler, so that it is now much more straightforward to explain how modern architectures work. This post is an attempt to explain directly how modern transformers work, and why, without some of the historical baggage.

<https://peterbloem.nl/blog/transformers>

By Peter Bloem

Transformers



The core new element in transformers is a process of “self-attention”.

The output y_i is a weighted average of the input vectors:

$$y_i = \sum_j w_{i,j} x_j$$

[home](#) / [blog](#) /

TRANSFORMERS FROM SCRATCH

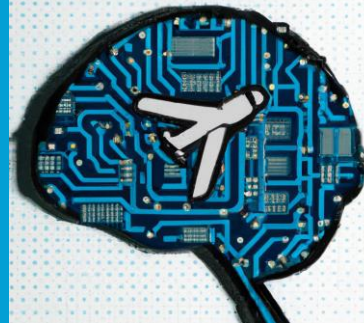
18 Aug 2019 · [code on github](#) · [video lecture](#)

Transformers are a very exciting family of machine learning architectures. Many good tutorials exist (e.g. [1, 2]) but in the last few years, transformers have mostly become simpler, so that it is now much more straightforward to explain how modern architectures work. This post is an attempt to explain directly how modern transformers work, and why, without some of the historical baggage.

<https://peterbloem.nl/blog/transformers>

By Peter Bloem

Transformers



The core new element in transformers is a process of “self-attention”.

The output $\mathbf{y}_i$ is a weighted average of the input vectors:

$$\mathbf{y}_i = \sum_j w_{i,j} \mathbf{x}_j$$

The weight here is not a parameter:

$$w'_{i,j} = \mathbf{x}_i^T \mathbf{x}_j, \quad w_{i,j} = \frac{\exp(w'_{i,j})}{\sum_j \exp(w'_{i,j})}$$

[home](#) / [blog](#) /

TRANSFORMERS FROM SCRATCH

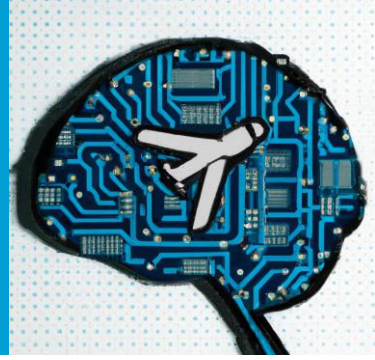
18 Aug 2019 · [code on github](#) · [video lecture](#)

Transformers are a very exciting family of machine learning architectures. Many good tutorials exist (e.g. [1, 2]) but in the last few years, transformers have mostly become simpler, so that it is now much more straightforward to explain how modern architectures work. This post is an attempt to explain directly how modern transformers work, and why, without some of the historical baggage.

<https://peterbloem.nl/blog/transformers>

By Peter Bloem

Transformers: self-attention



The core new element in transformers is a process of “self-attention”.

The output $\mathbf{y}_i$ is a weighted average of the input vectors:

$$\mathbf{y}_i = \sum_j w_{i,j} \mathbf{x}_j$$

The weight here is not a parameter:

$$w'_{i,j} = \mathbf{x}_i^T \mathbf{x}_j,$$

$$w_{i,j} = \frac{\exp(w'_{i,j})}{\sum_j \exp(w'_{i,j})} \left. \vphantom{\frac{\exp(w'_{i,j})}{\sum_j \exp(w'_{i,j})}} \right\} \text{Soft-max function}$$

[home](#) / [blog](#) /

TRANSFORMERS FROM SCRATCH

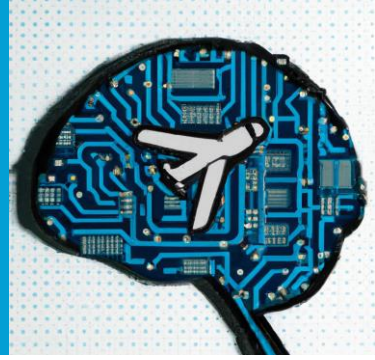
18 Aug 2019 · [code on github](#) · [video lecture](#)

Transformers are a very exciting family of machine learning architectures. Many good tutorials exist (e.g. [1, 2]) but in the last few years, transformers have mostly become simpler, so that it is now much more straightforward to explain how modern architectures work. This post is an attempt to explain directly how modern transformers work, and why, without some of the historical baggage.

<https://peterbloem.nl/blog/transformers>

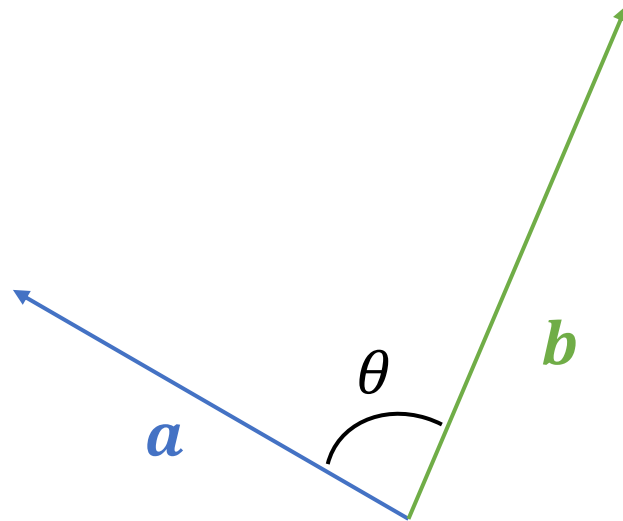
By Peter Bloem

Transformers: self-attention

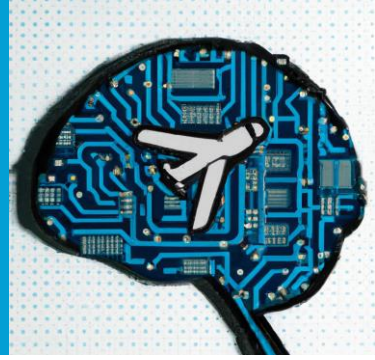


The dot product between two vectors captures similarity:

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos(\theta)$$



Transformers: queries, keys, values

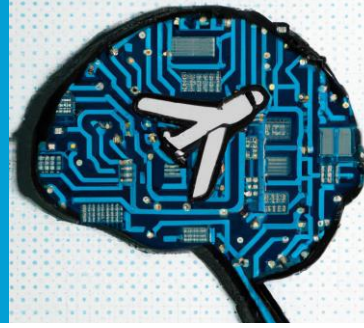


Vectors $\mathbf{x}_i$ play three roles:

1. In the weight, it can be the “**query**”
2. In the weight, it can be the “**key**”
3. In the weighted sum, it can be the “**value**”

$$\mathbf{y}_i = \sum_j w_{i,j} \mathbf{x}_j$$
$$w'_{i,j} = \mathbf{x}_i^T \mathbf{x}_j$$

Transformers: queries, keys, values

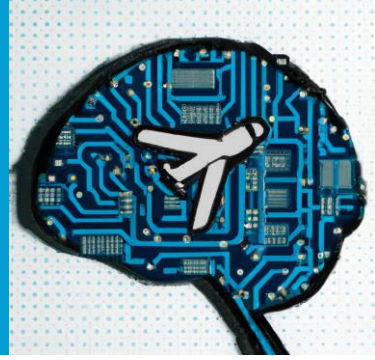


To accommodate the difference between these three roles, input vectors are linearly transformed before using them, with (learned) weight matrices W_q, W_k, W_v :

$$\mathbf{q}_i = W_q \mathbf{x}_i, \mathbf{k}_j = W_k \mathbf{x}_j, \mathbf{v}_j = W_v \mathbf{x}_j$$

$$\mathbf{y}_i = \sum_j w_{i,j} \mathbf{v}_j$$
$$w'_{i,j} = \mathbf{q}_i^T \mathbf{k}_j$$

Transformers: scaling the weight

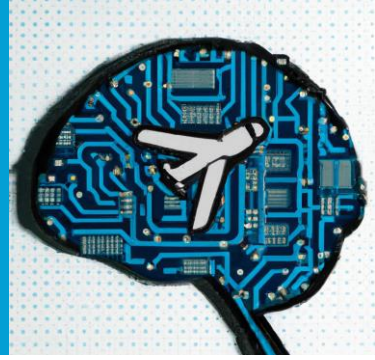


The dot-product is scaled to prevent large values in the softmax function:

$$w'_{i,j} = \frac{\mathbf{q}_i^T \mathbf{k}_j}{\sqrt{D}}$$

Where D is the dimensionality of the vector, $\mathbf{x}_i \in \mathbb{R}^D$.

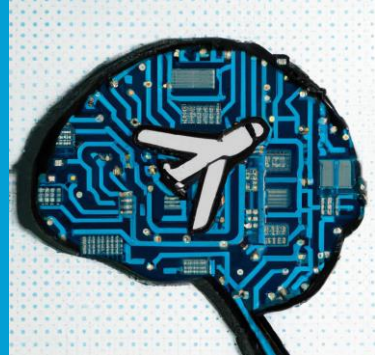
Transformers: multi-head attention



Allow for multiple attention mechanisms by using multiple weight matrices, here indexed by r :

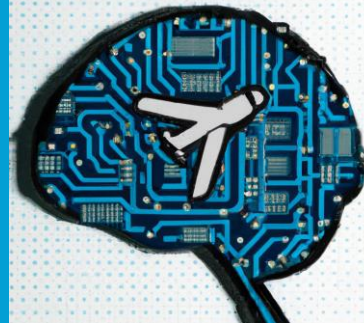
$$W_q^r, W_k^r, W_v^r$$

Transformers: order



Until here, the transformer actually operates completely on **sets**, with no regards for the order of the inputs!

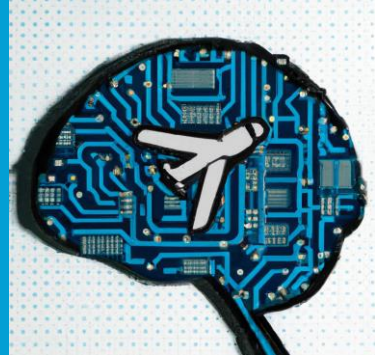
Transformers: order



Until here, the transformer actually operates completely on **sets**, with no regards for the order of the inputs!

To provide information on the order of inputs, transformers add a position vector $\mathbf{p}$ to $\mathbf{x}$.

Transformers: order

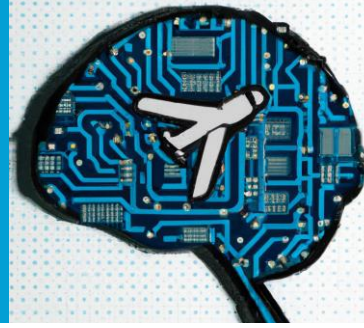


Until here, the transformer actually operates completely on **sets**, with no regards for the order of the inputs!

To provide information on the order of inputs, transformers add a position vector $\mathbf{p}$ to $\mathbf{x}$.

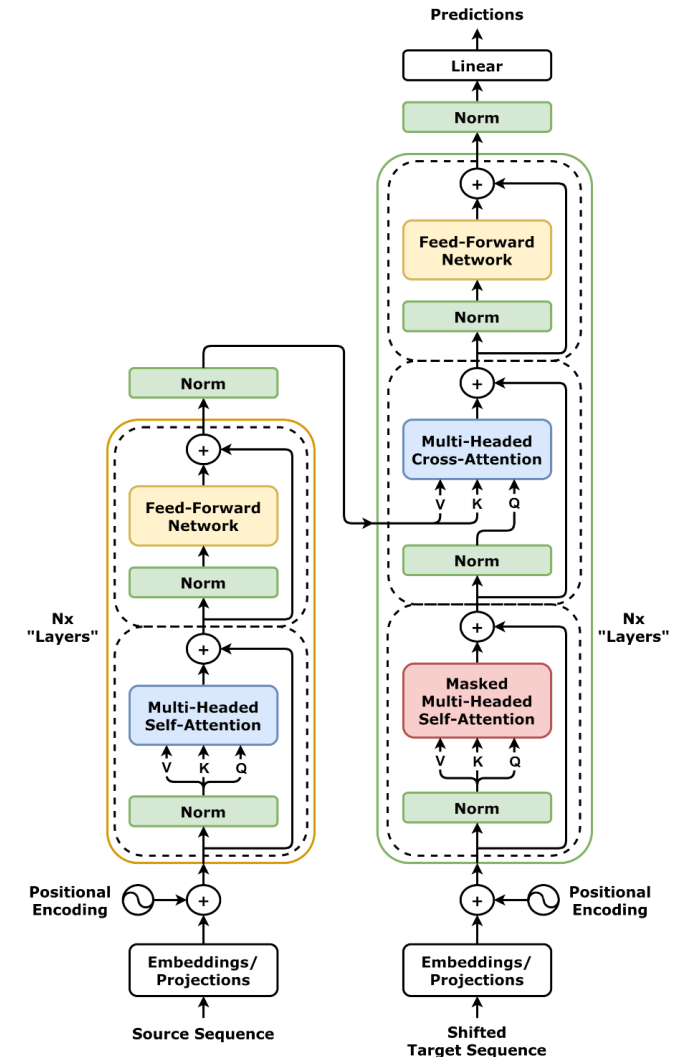
This position vector can be pre-determined (“position encoding”) or learned (“position embedding”).

Transformer: architecture

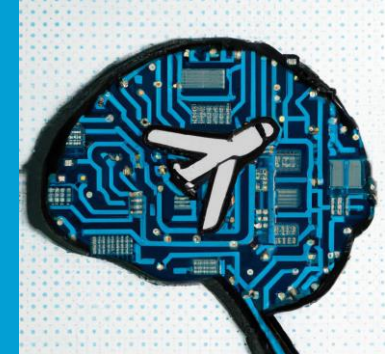


In a typical transformer architecture, the self-attention block is combined with:

- layer normalization
- feed-forward neural networks
- masks excluding future words



Transformers: synthesis



Advantages



...



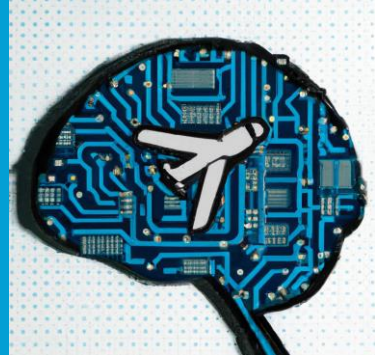
...

Disadvantages



...

Transformers: synthesis

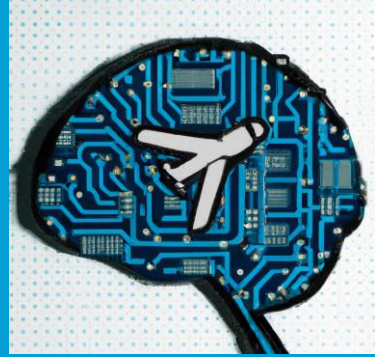


Advantages

- ✓ Extremely powerful architecture with state-of-the-art performance – especially on language tasks, but even for vision and beyond
- ✓ Although it involves many computational operations, they are easy to parallelize and hence run fast on GPUs.

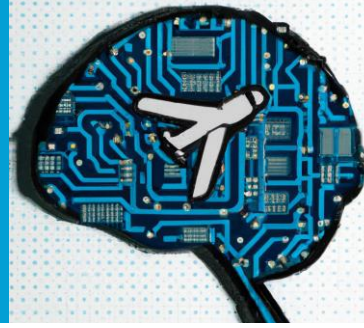
Disadvantages

- ✗ The memory requirements are large and grow with an increasing time range.

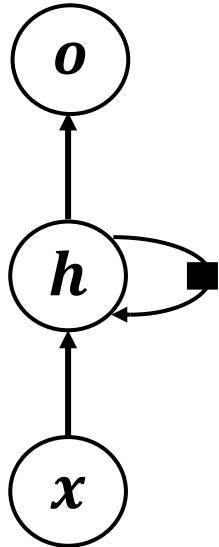


Recurrency

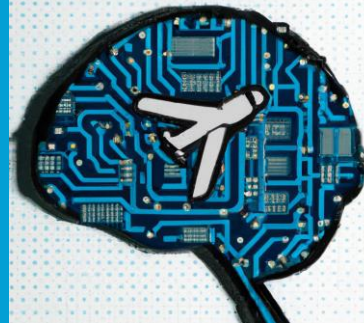
Recurrent neural networks



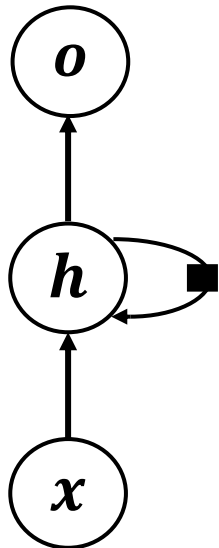
Recurrent neural networks can parse sequences of any length, with “parameter sharing” over time:



Recurrent neural networks



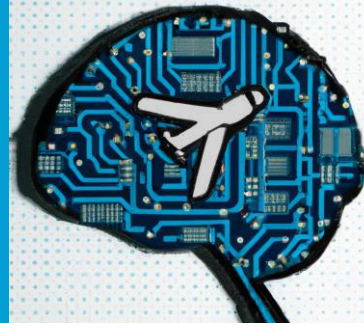
Recurrent neural networks can parse sequences of any length, with “parameter sharing” over time:



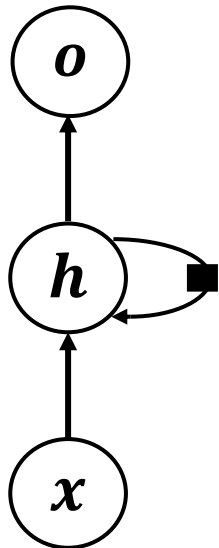
$$\mathbf{h}_t = f(U\mathbf{x}_t + \mathbf{b} + W\mathbf{h}_{t-1})$$

$$\mathbf{o}_t = g(\mathbf{c} + V\mathbf{h}_t)$$

Recurrent neural networks



Recurrent neural networks can parse sequences of any length, with “parameter sharing” over time:

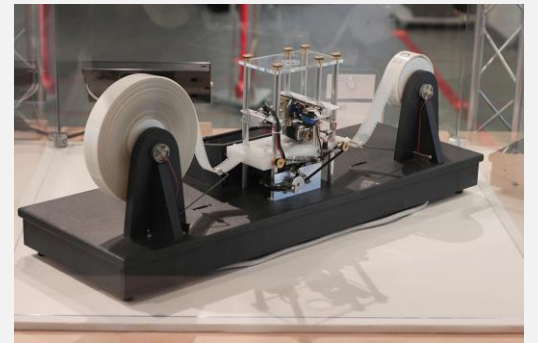


$$\mathbf{h}_t = f(U\mathbf{x}_t + \mathbf{b} + W\mathbf{h}_{t-1})$$

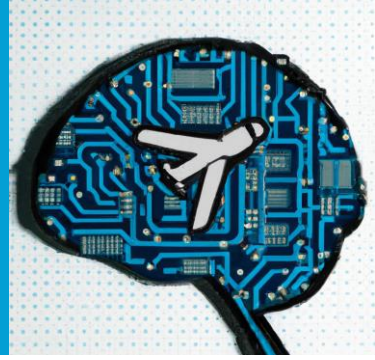
$$\mathbf{o}_t = g(\mathbf{c} + V\mathbf{h}_t)$$

Note:

*This form of recurrent neural network is **Turing complete**, i.e., it can implement any computer algorithm.*

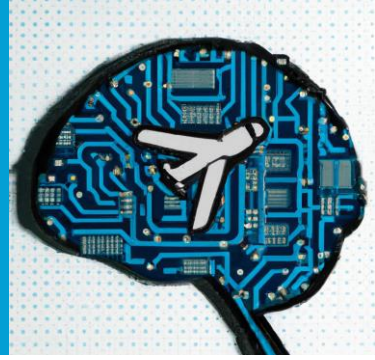


Difficult?



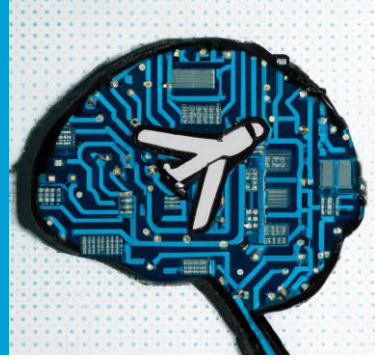
Try it out yourself. Watch the following numbers carefully. Then I will ask you the next number.

Difficult?



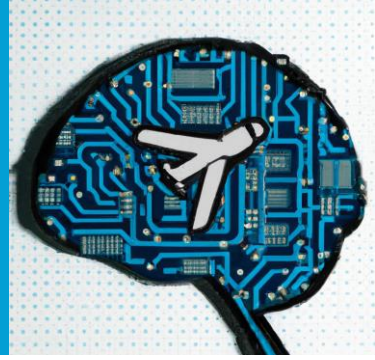
112

Difficult?



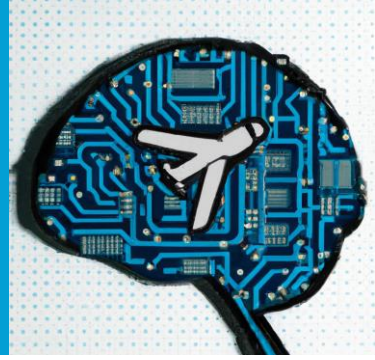
118

Difficult?



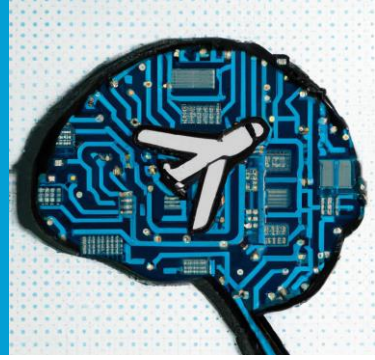
132

Difficult?



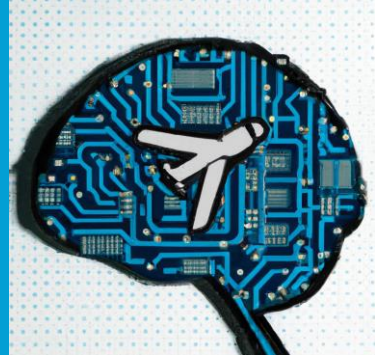
129

Difficult?



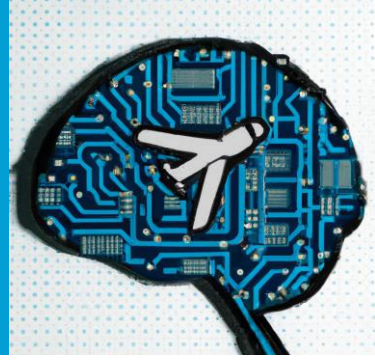
121

Difficult?



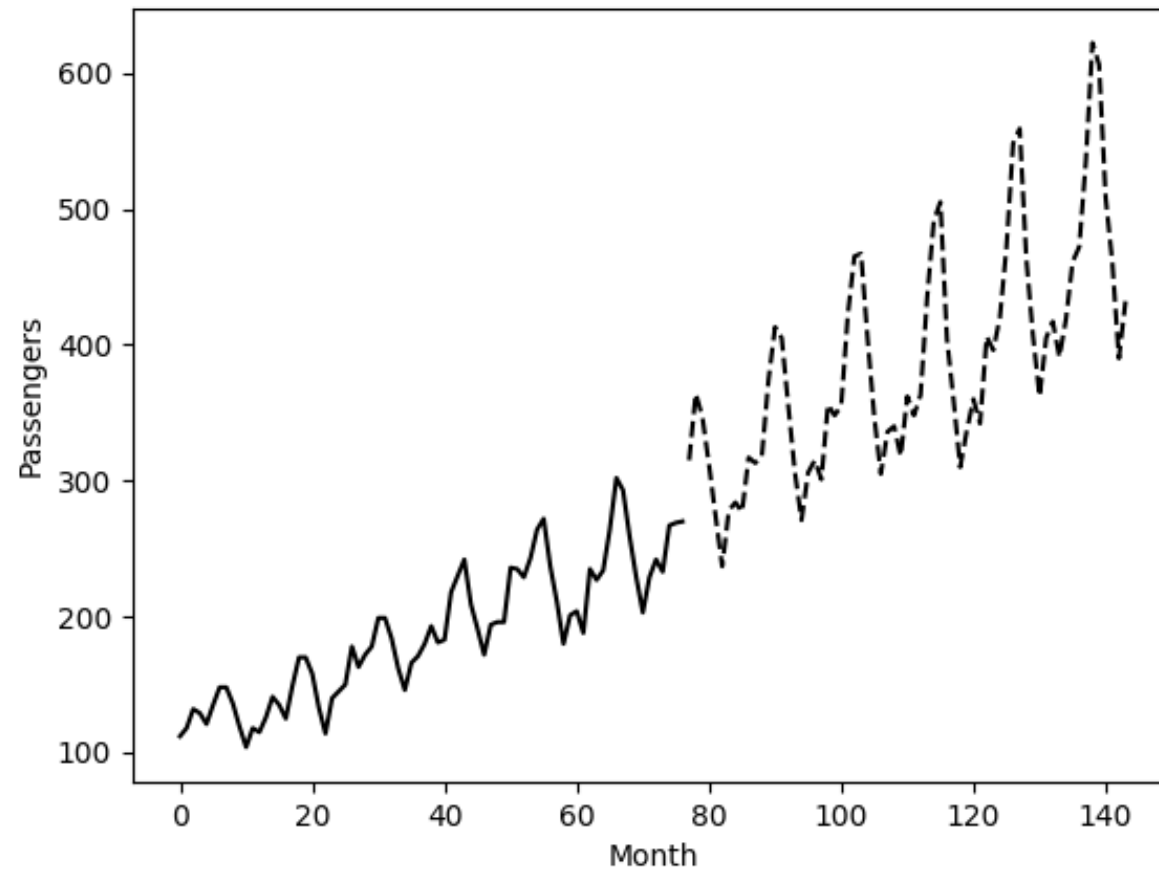
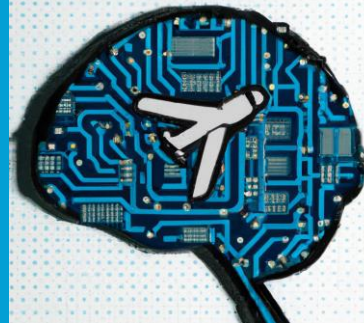
?

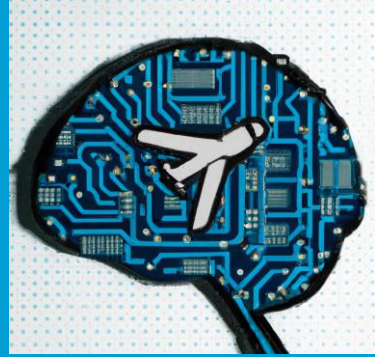
The answer:



135

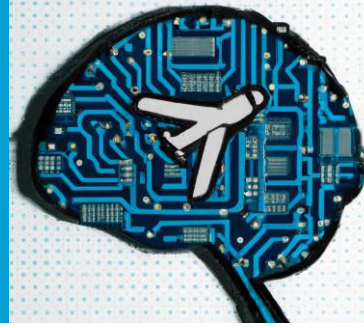
Easier with a time window?



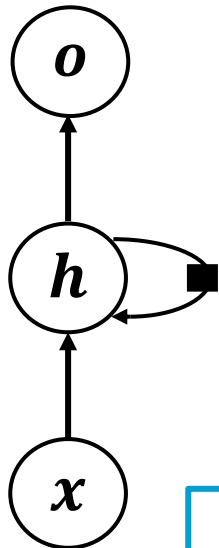


Training RNNs

Recurrent neural networks



Recurrent neural networks can parse sequences of any length, with “parameter sharing” over time:

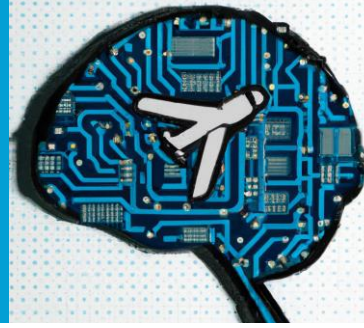


$$\mathbf{h}_t = f(U\mathbf{x}_t + \mathbf{b} + W\mathbf{h}_{t-1})$$

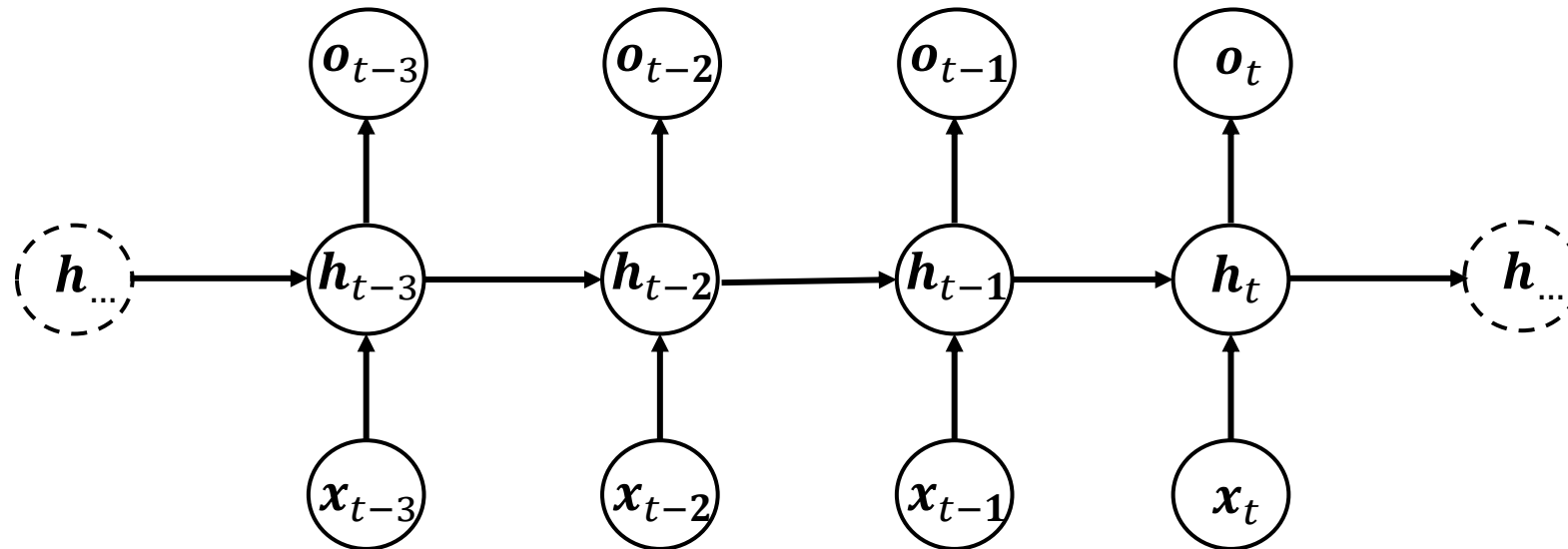
$$\mathbf{o}_t = g(\mathbf{c} + V\mathbf{h}_t)$$

Is recurrency a problem for backpropagation? Why / why not?

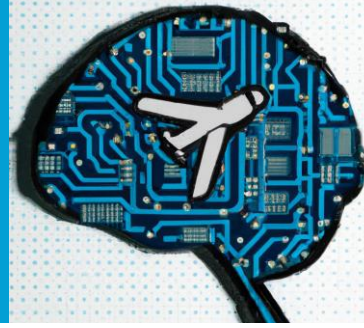
Recurrent neural networks



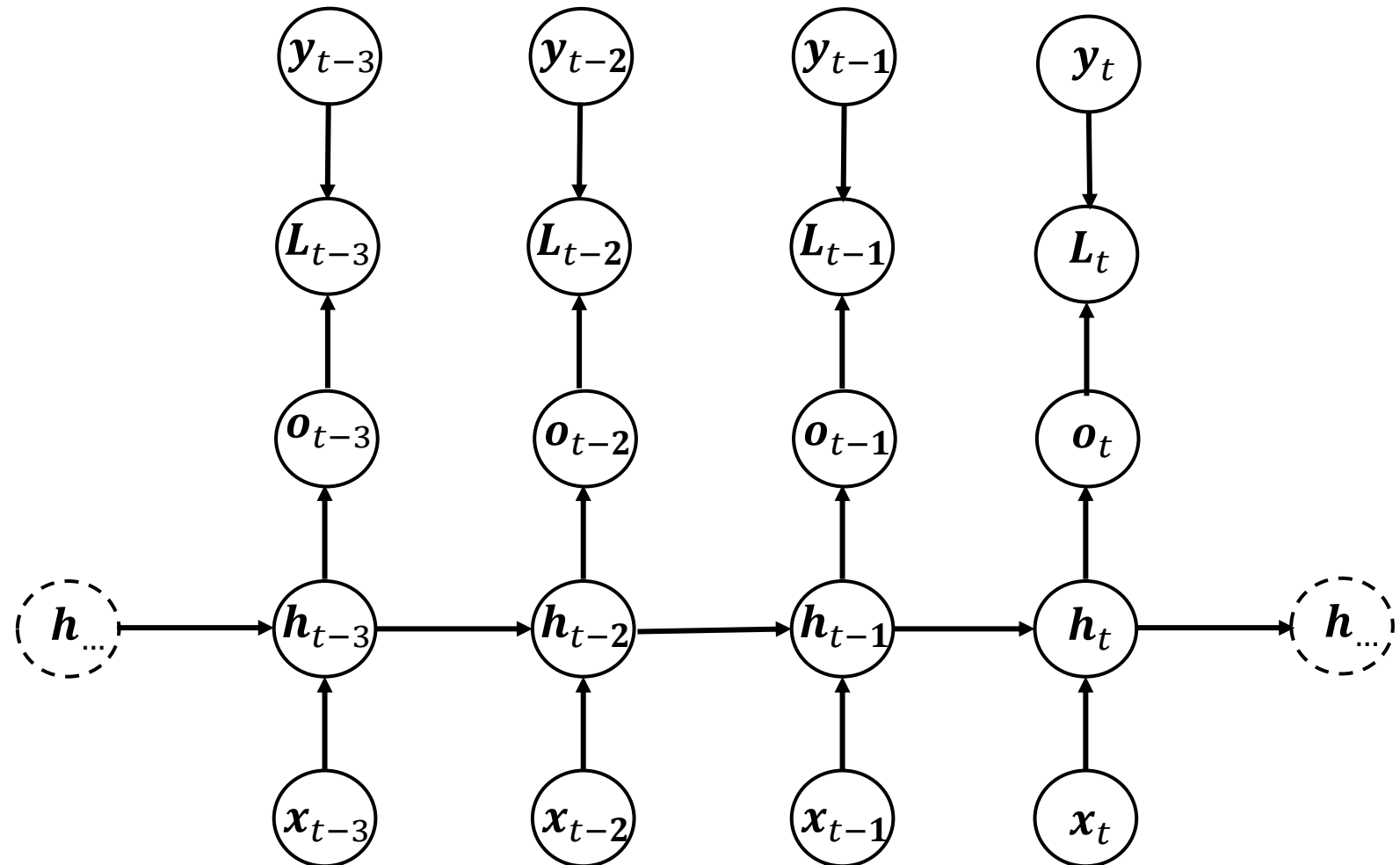
“Computational graph” – unrolling the network over time.



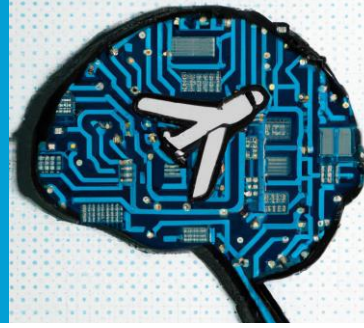
Backpropagation through time (BPTT)



Computational graph
with losses L_t and
targets y_t .

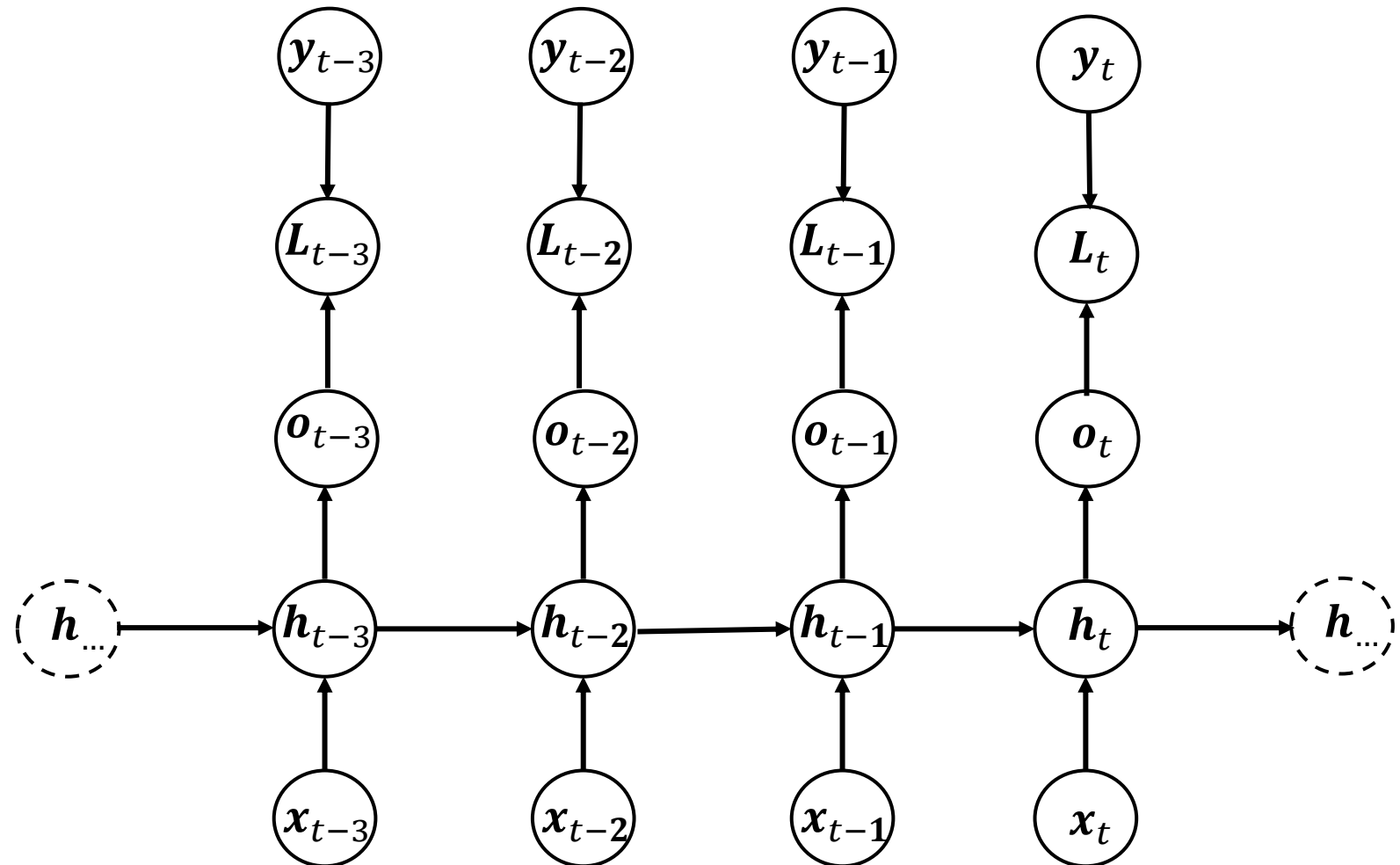


Backpropagation through time (BPTT)

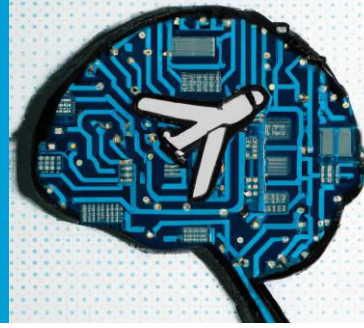


Computational graph with losses L_t and targets y_t .

The arrows show the forward pass.



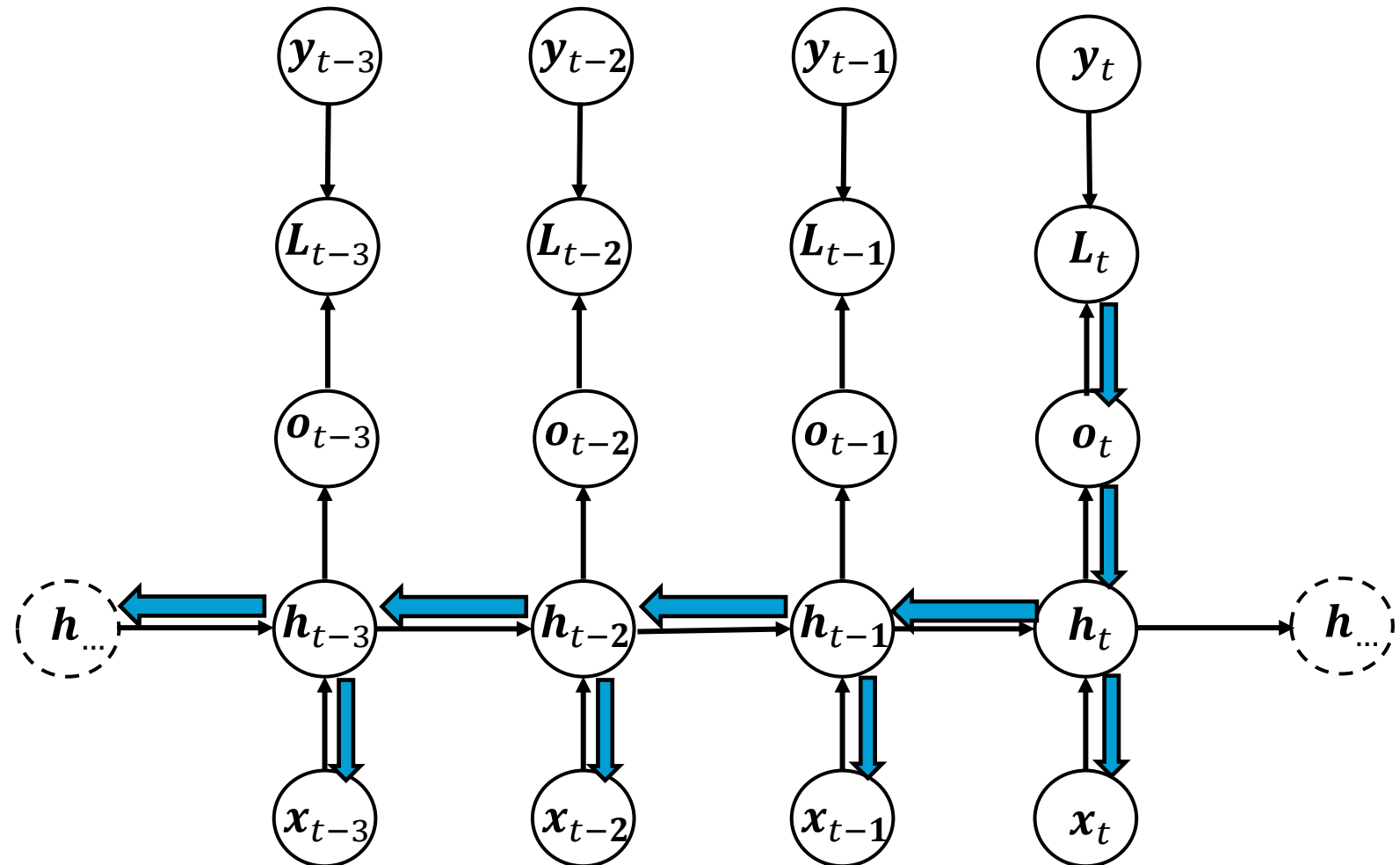
Backpropagation through time (BPTT)



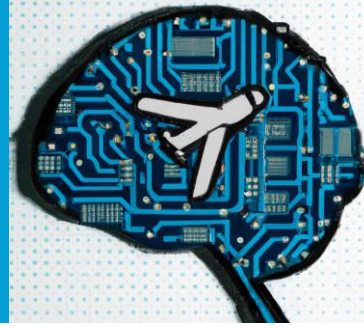
Computational graph with losses L_t and targets y_t .

The arrows show the forward pass.

Backward pass for L_t



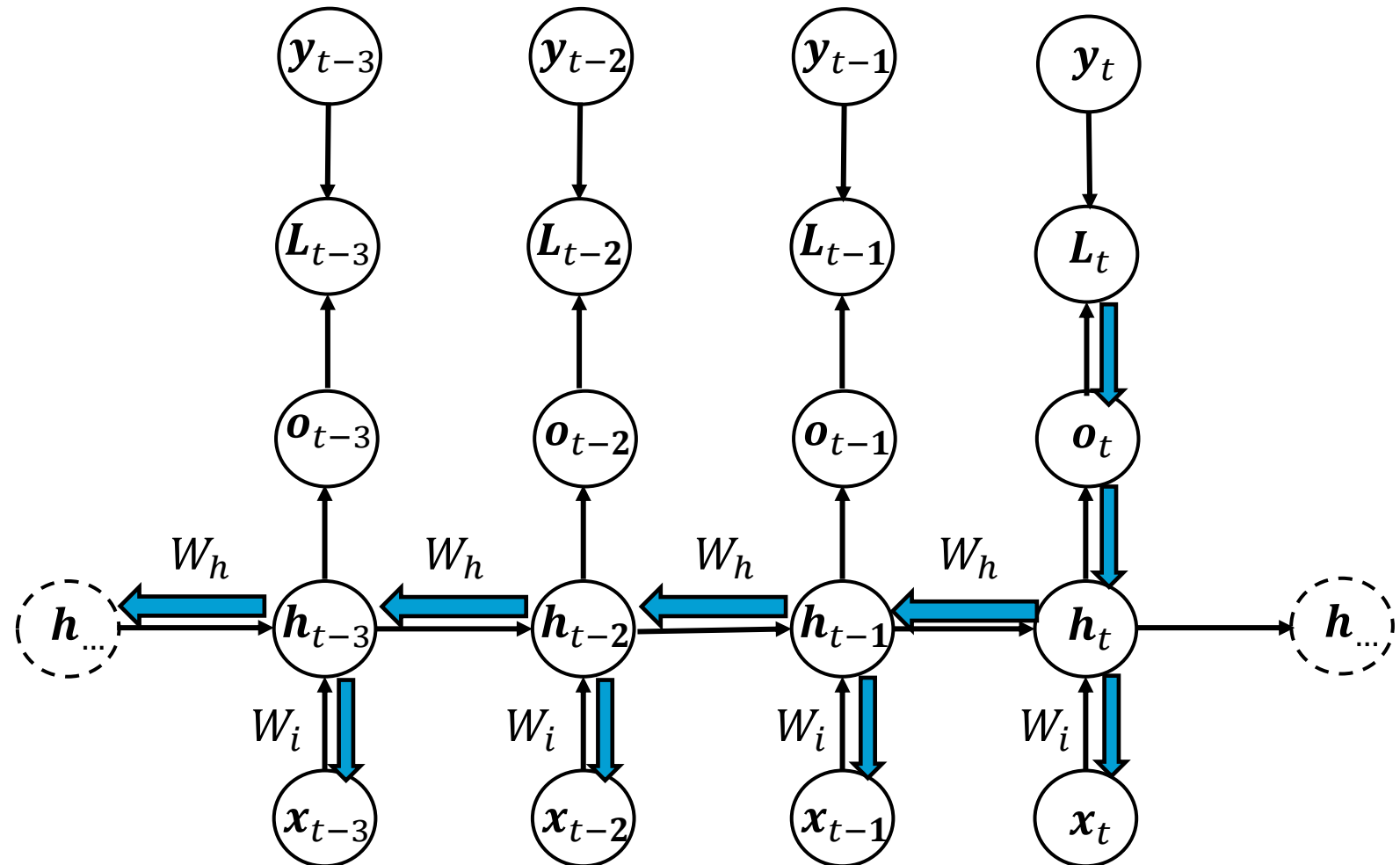
Backpropagation through time (BPTT)



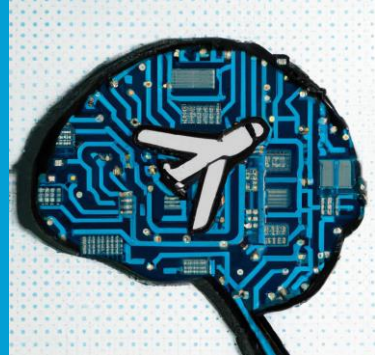
Computational graph with losses L_t and targets y_t .

The arrows show the forward pass.

Backward pass for L_t

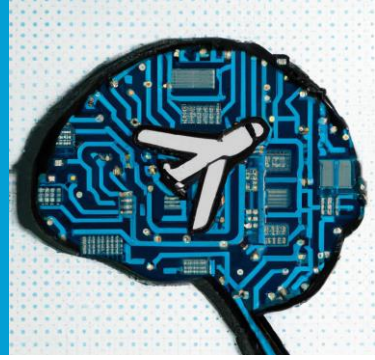


Backpropagation through time (BPTT)



Do you foresee any challenges with BPTT?

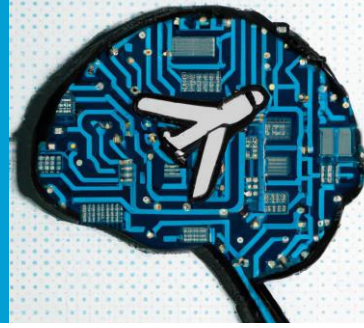
Backpropagation through time (BPTT)



Do you foresee any challenges with BPTT?

- Long time sequences correspond to very deep graphs, with associated problems of gradient vanishing / explosion.
- Cutting off the time window will prevent the network from learning long-term sequence patterns.

Challenge of long-term dependencies



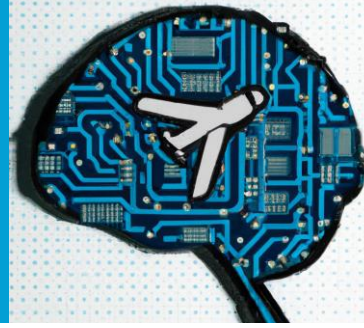
Let's take a simplified version of an RNN:

$$h_t = W h_{t-1}$$

Propagating over time gives:

$$h_{t+2} = W h_{t+1} = W(W h_t) = W(W(W h_{t-1})) = W^3 h_{t-1}$$

Challenge of long-term dependencies



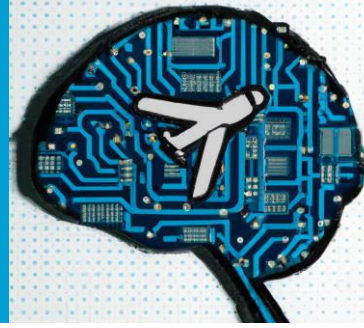
So, after n steps:

$$\mathbf{h}_{t+n} = W^n \mathbf{h}_t$$

Assuming the weight matrix admits an Eigendecomposition of $W = Q^T D Q$, with Q an orthogonal matrix and D a diagonal matrix with W 's eigenvalues λ :

$$\mathbf{h}_{t+n} = Q^T D^n Q \mathbf{h}_t$$

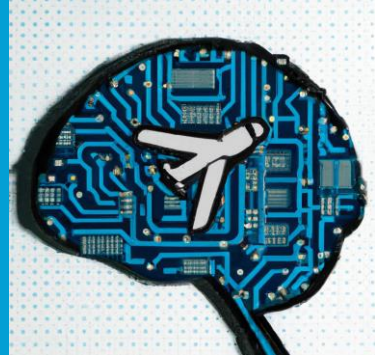
Challenge of long-term dependencies



Hence, the eigenvalues are raised to the power of n , meaning that eigenvalues smaller than one decay to zero and eigenvalues larger than one will explode.

Components of $\mathbf{h}_t$ not aligned with the largest eigenvector will be discarded.

Alternative learning method

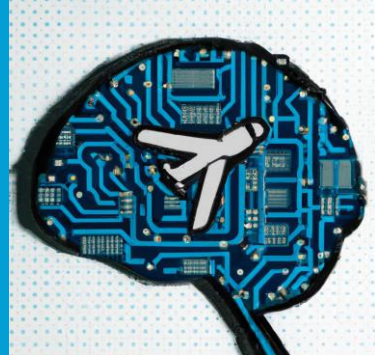


Meta-heuristic optimization methods

- Evolutionary algorithms
- Particle Swarm Optimization
- ...



Alternative learning method



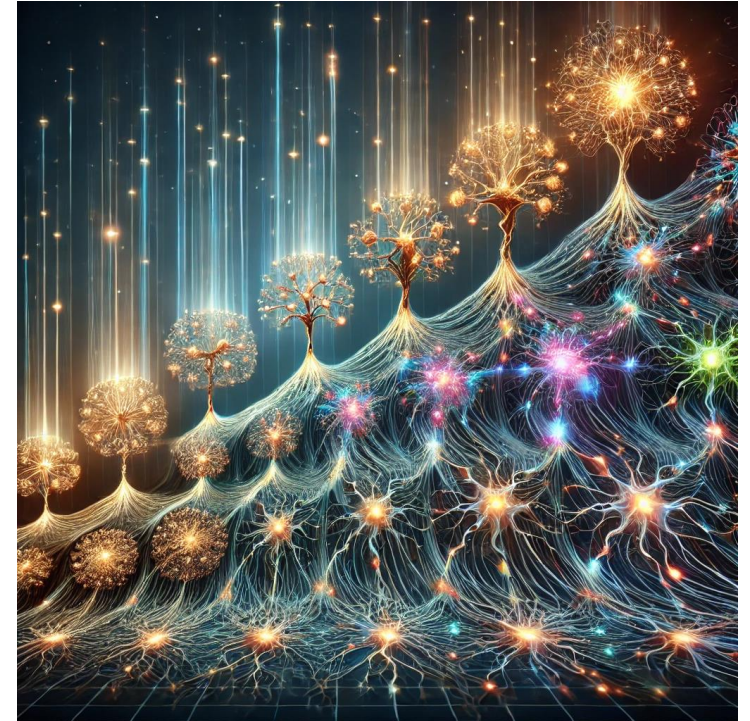
Meta-heuristic optimization methods

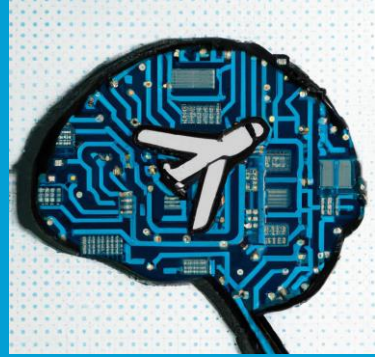
Advantage:

✓ No problem with vanishing gradients.

Disadvantages:

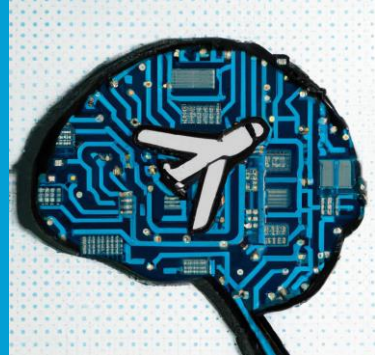
- ✗ Time-intensive
- ✗ Not very suitable for fine-tuned dynamics
- ✗ Less performance in high-dimensional search spaces





Alternative memory types

Leaky neurons



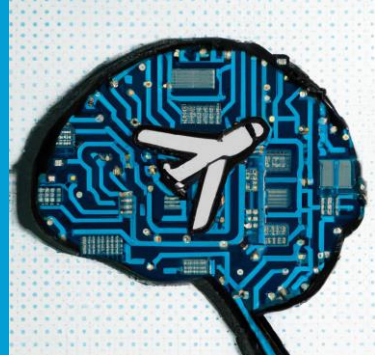
Another form of memory is to have “neural inertia”, nowadays called leaky neurons:

$$\mathbf{h}_t = \alpha \mathbf{h}_{t-1} + (1 - \alpha) f(W\mathbf{x}_t + \mathbf{b})$$

This makes the hidden activation a low-passed version of the past activation values.



Leaky neurons



Another form of memory is to have “neural inertia”, nowadays called leaky neurons:

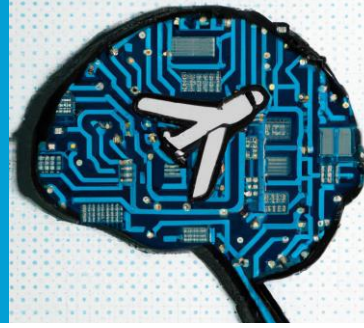
$$\mathbf{h}_t = \alpha \mathbf{h}_{t-1} + (1 - \alpha) f(W\mathbf{x}_t + \mathbf{b})$$

This makes the hidden activation a low-passed version of the past activation values.

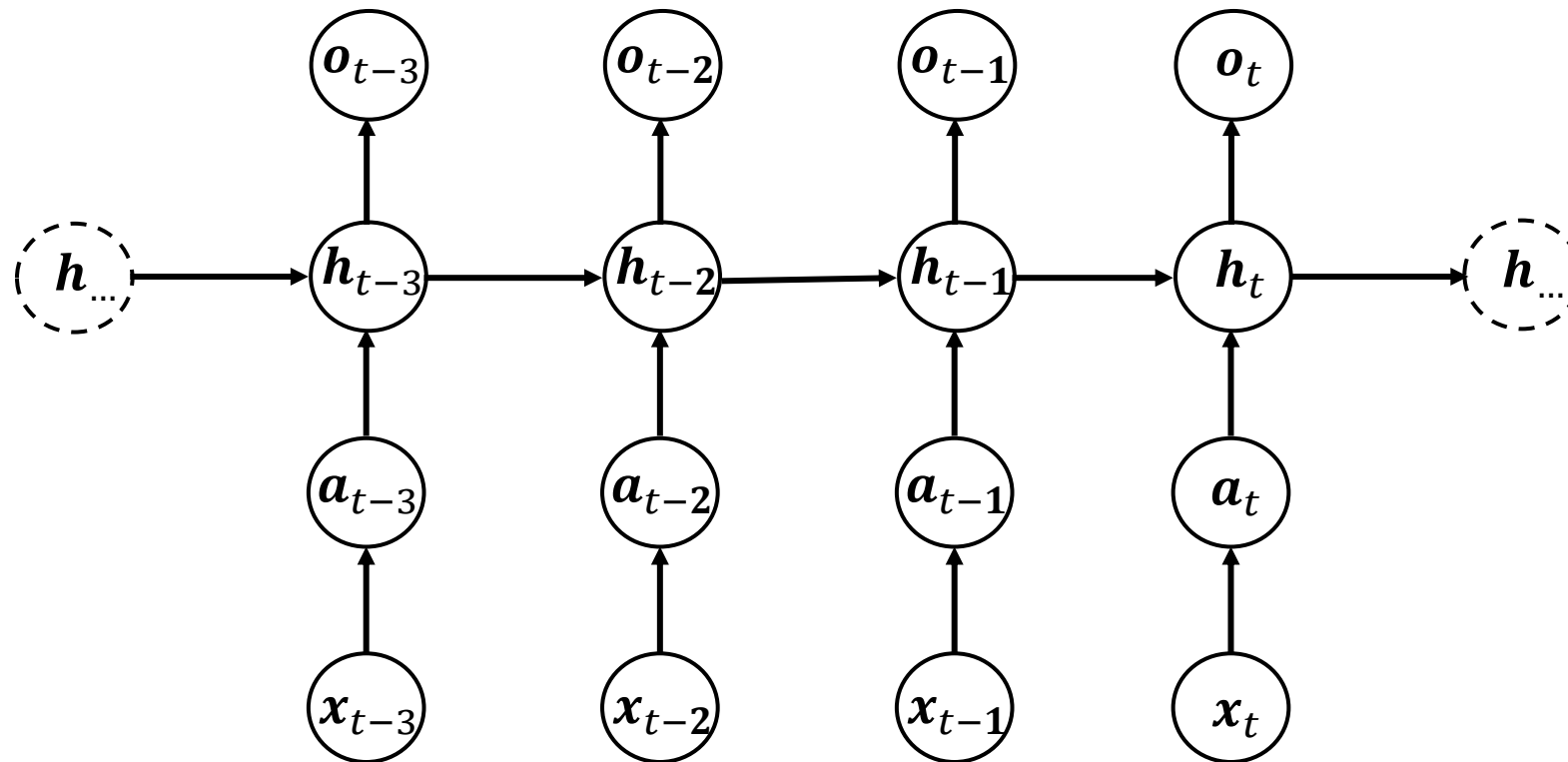
Having neurons with different leaks allows to exploit information at different time scales.



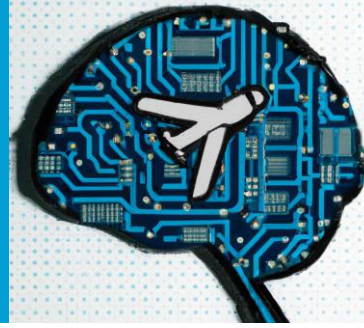
Leaky neurons



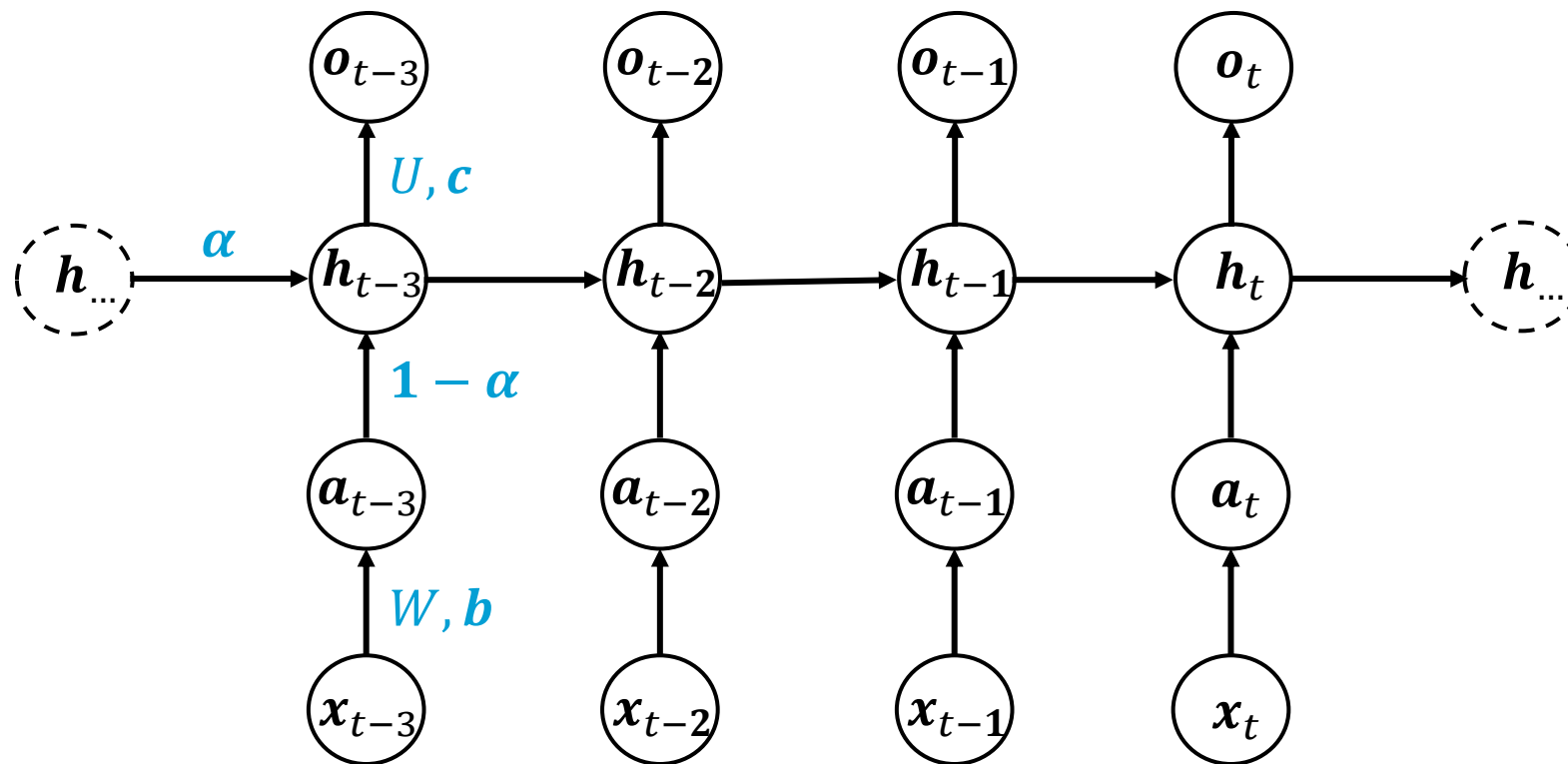
The leaks α can be learned. Below $\mathbf{a}_t = f(W\mathbf{x}_t + \mathbf{b})$:



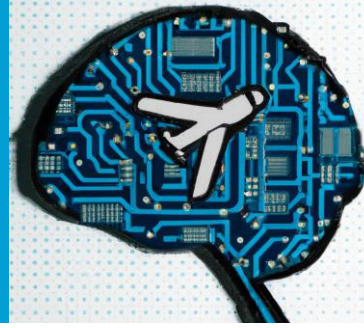
Leaky neurons



The leaks α can be learned. Below $\mathbf{a}_t = f(W\mathbf{x}_t + \mathbf{b})$:



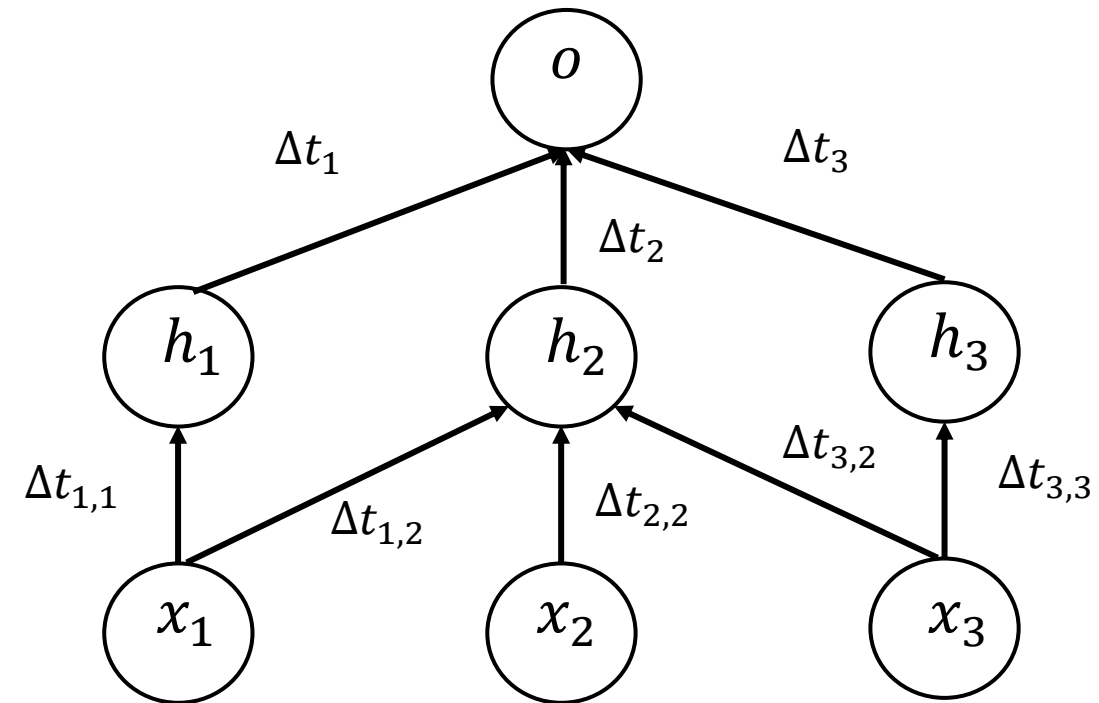
Delays



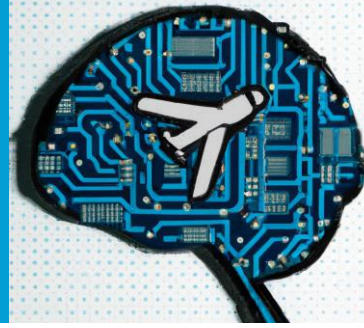
Time delays on the connections:

$$h_{j,t} = \sum_i w_{i,j} x_i(t - \Delta t_{i,j}).$$

Having different time delays allows for exploiting information at different time scales.



Delays

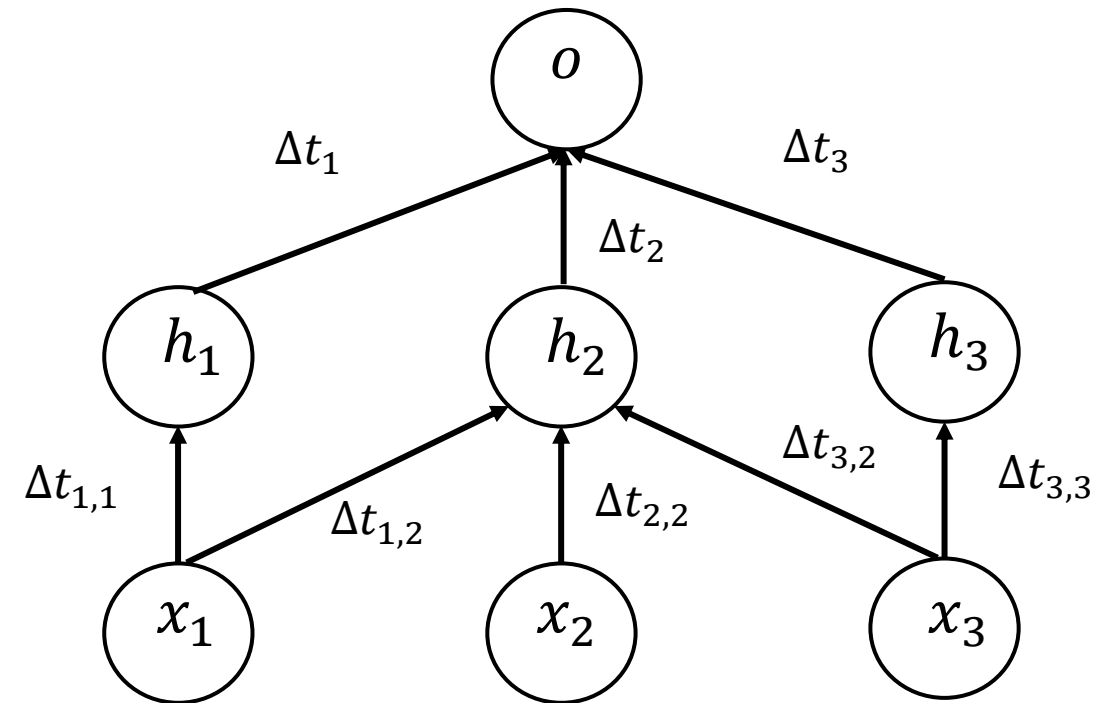


Time delays on the connections:

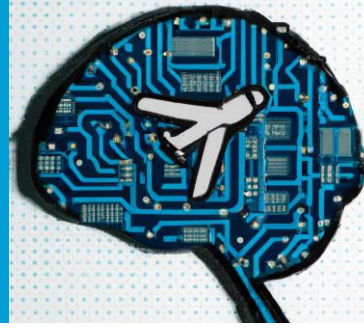
$$h_{j,t} = \sum_i w_{i,j} x_i(t - \Delta t_{i,j}).$$

Having different time delays allows for exploiting information at different time scales.

How to learn the weights?



Delays



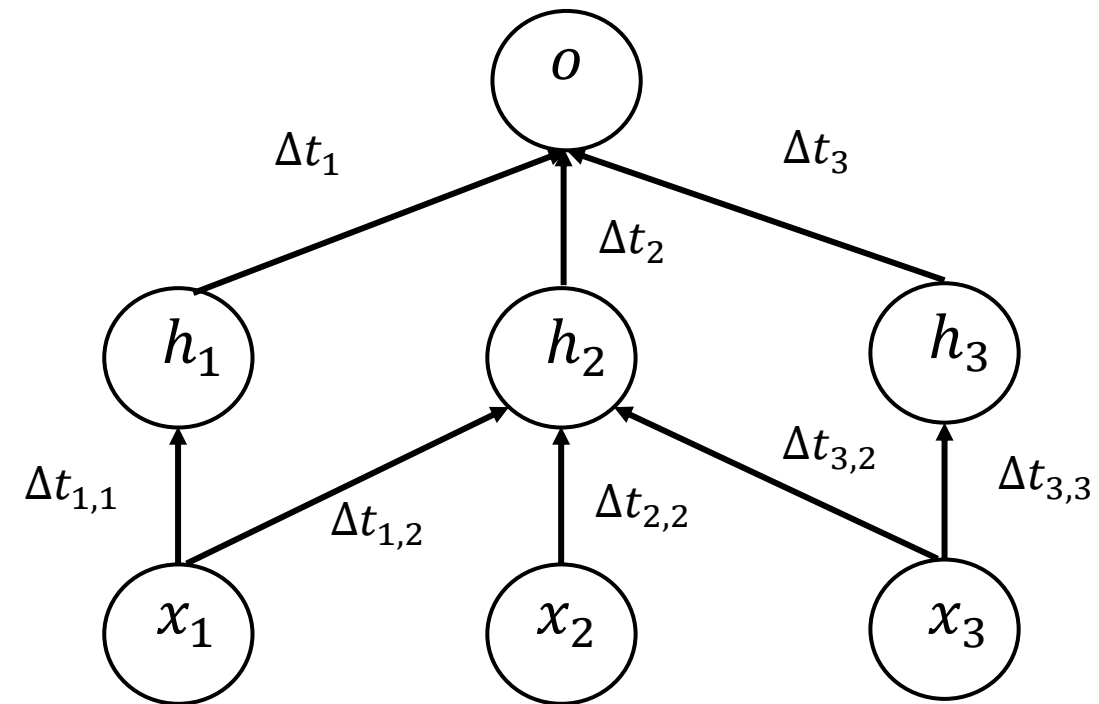
Time delays on the connections:

$$h_{j,t} = \sum_i w_{i,j} x_i(t - \Delta t_{i,j}).$$

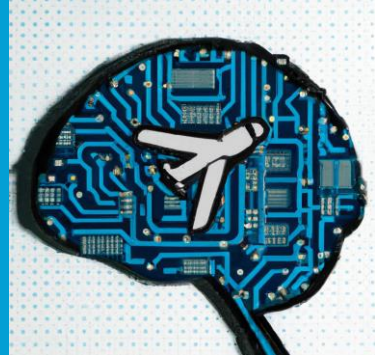
Having different time delays allows for exploiting information at different time scales.

How to learn the weights?

How to learn the time delays?

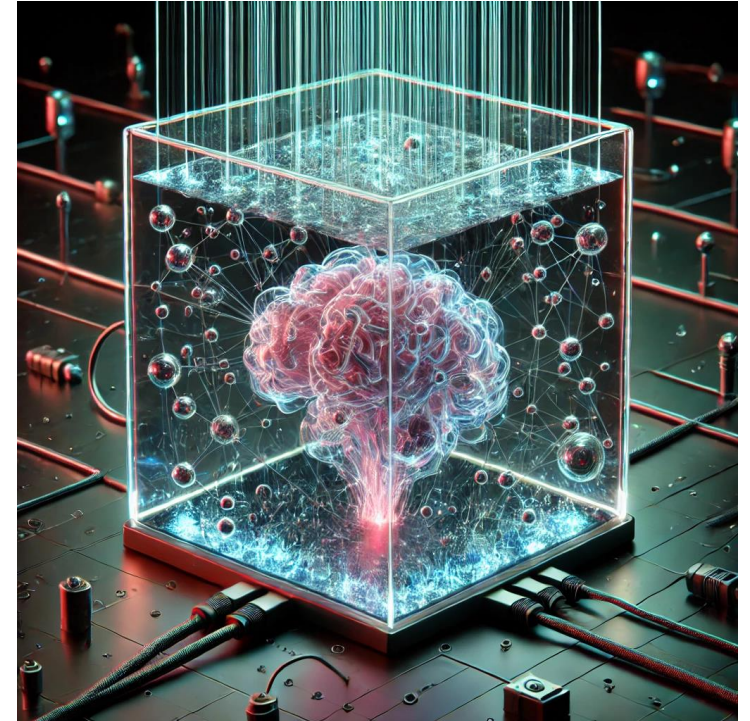


Reservoir computing



Circumvent the training problem by:

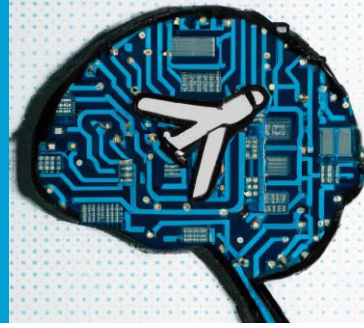
1. Creating a “random” recurrent neural network that has rich internal dynamics.
2. Only training a feedforward read-out network.



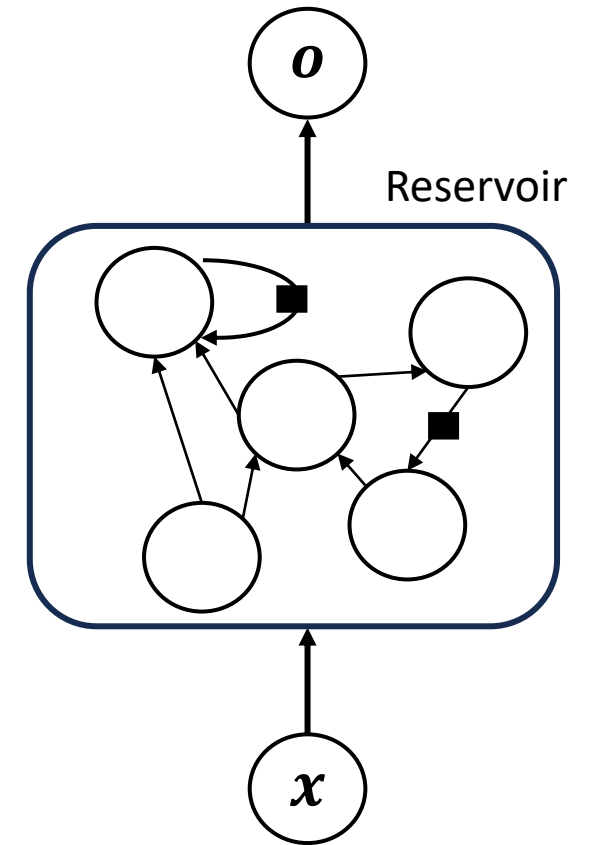
Echo state network: Jaeger, H., & Haas, H. (2004). Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication. *Science*, 304(5667), 78-80.

Liquid state machine: Maass, W., Natschläger, T., & Markram, H. (2002). Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural computation*, 14(11), 2531-2560.

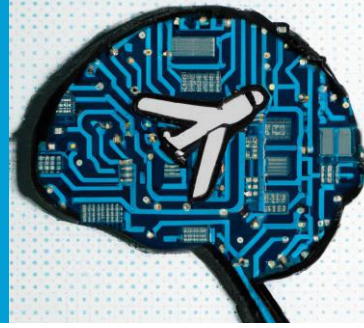
Reservoir computing



How to construct a reservoir to have rich enough dynamics?



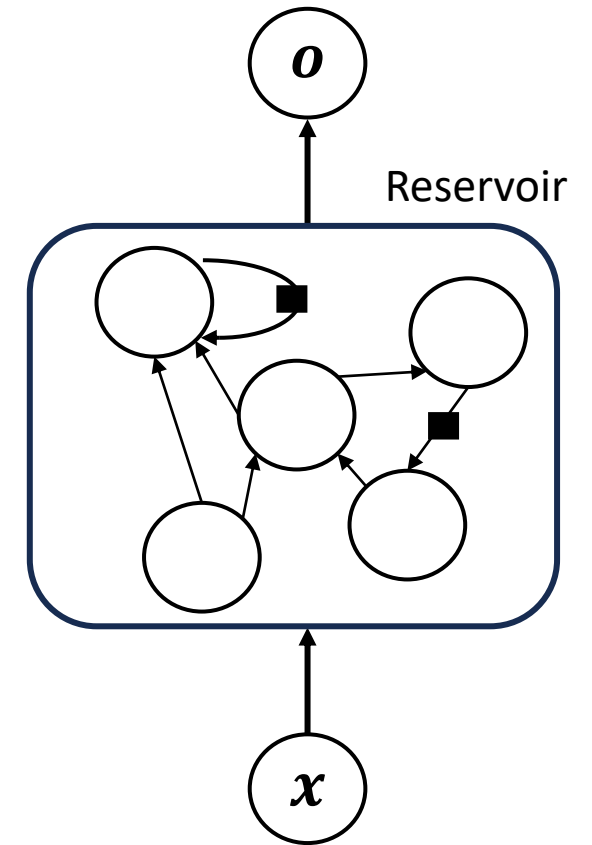
Reservoir computing



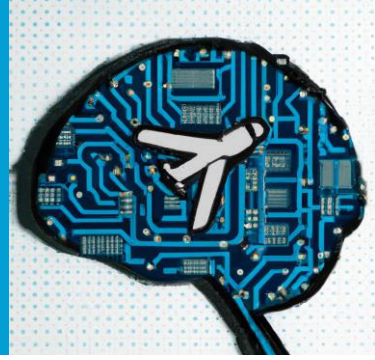
Edge of stability: Design the Jacobian of the state transitions $J_t = \frac{\partial s_t}{\partial s_{t-1}}$ with a given spectral radius:
 $\rho(A) = \max\{|\lambda_1|, |\lambda_2|, \dots, |\lambda_n|\}$

- Create a matrix W with elements in $[-1,1]$
- Then scale it with a desired spectral radius ρ^* :

$$W_{\text{res}} = \rho^* \frac{W}{\rho(W)}$$



Reservoir computing



Use a meta-heuristic (black-box) optimization algorithm

- Particle Swarm Optimization
- Evolutionary algorithms
- ...

Advantage:

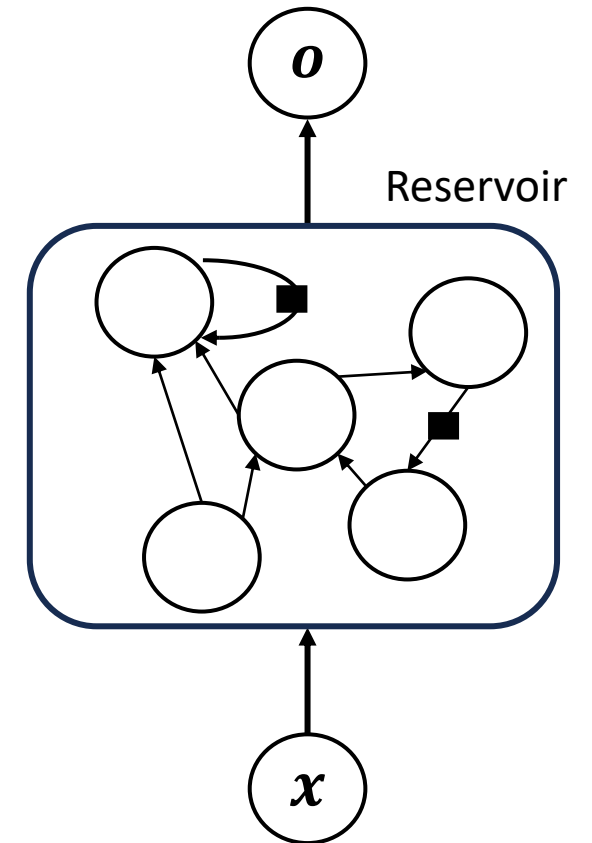


Dynamics for a specific task.

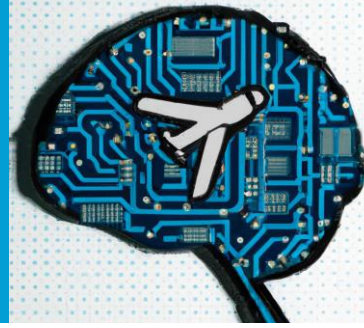
Disadvantage:



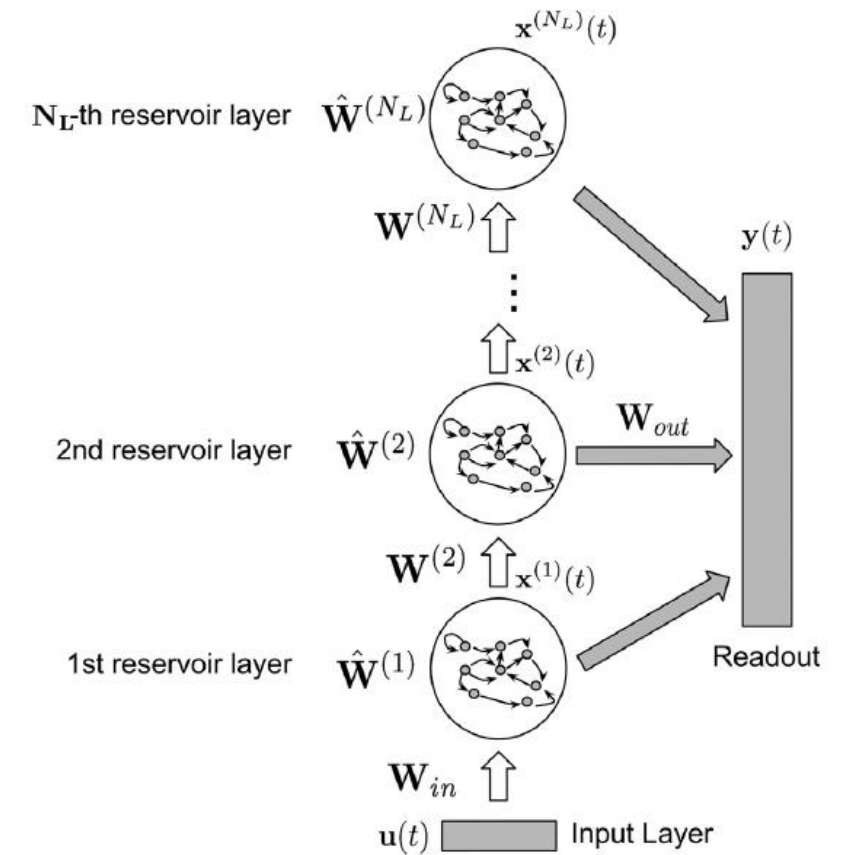
Substantial extra training time.



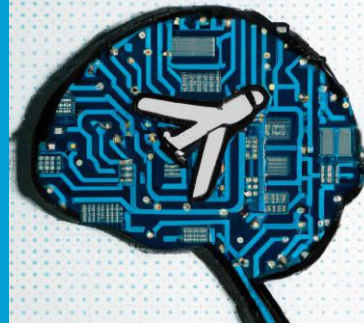
Reservoir computing



Deep ESNs

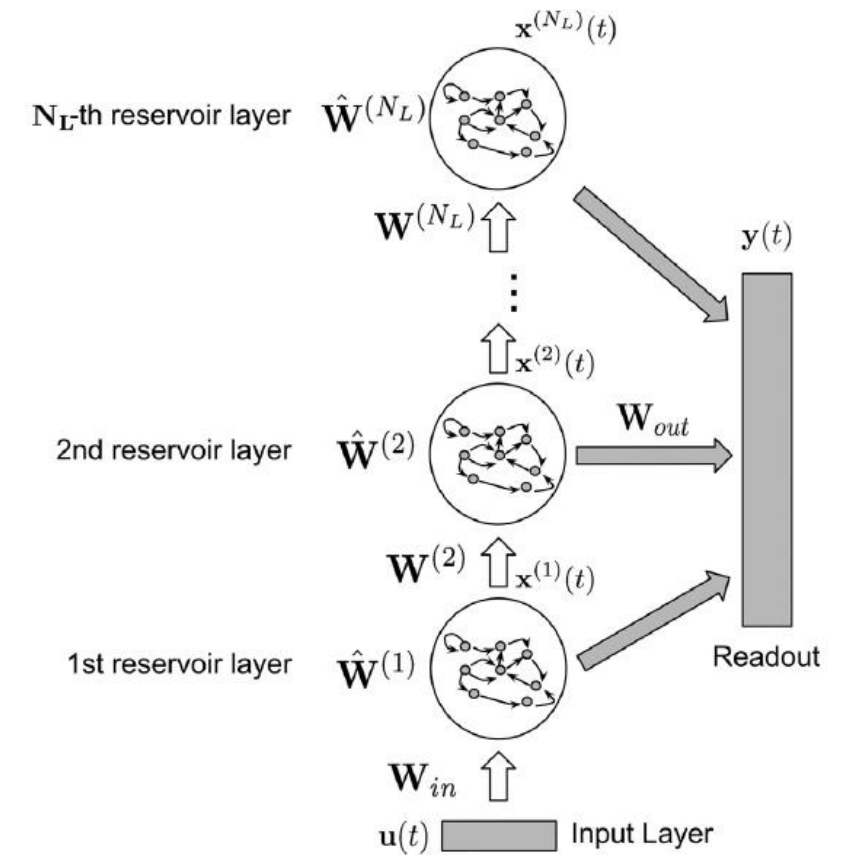


Reservoir computing

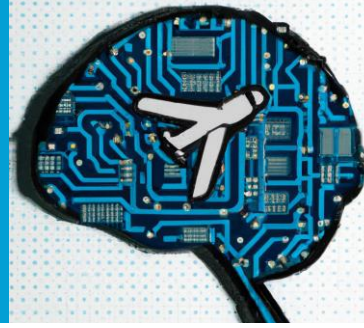


Deep ESNs

- Multiple-time-scale dynamics

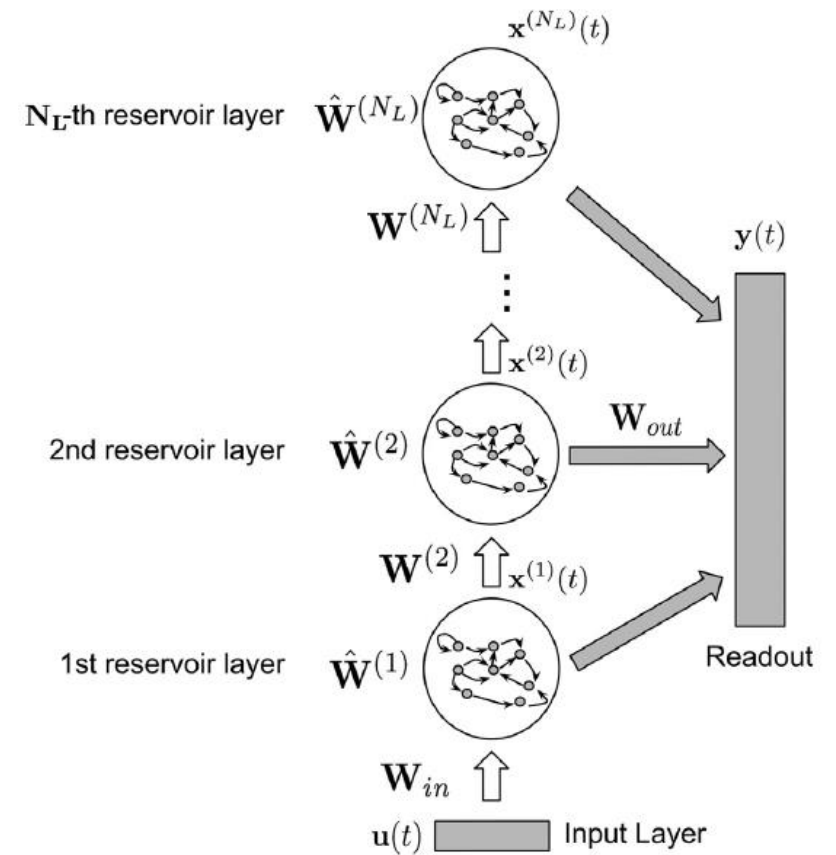


Reservoir computing

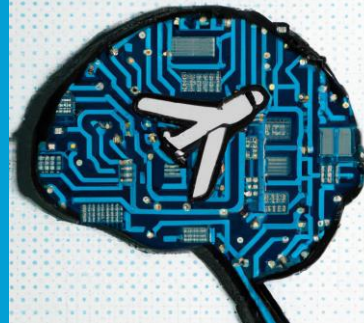


Deep ESNs

- Multiple-time-scale dynamics
- Richness of reservoir states: higher layers have less contractive dynamics



Reservoir computing

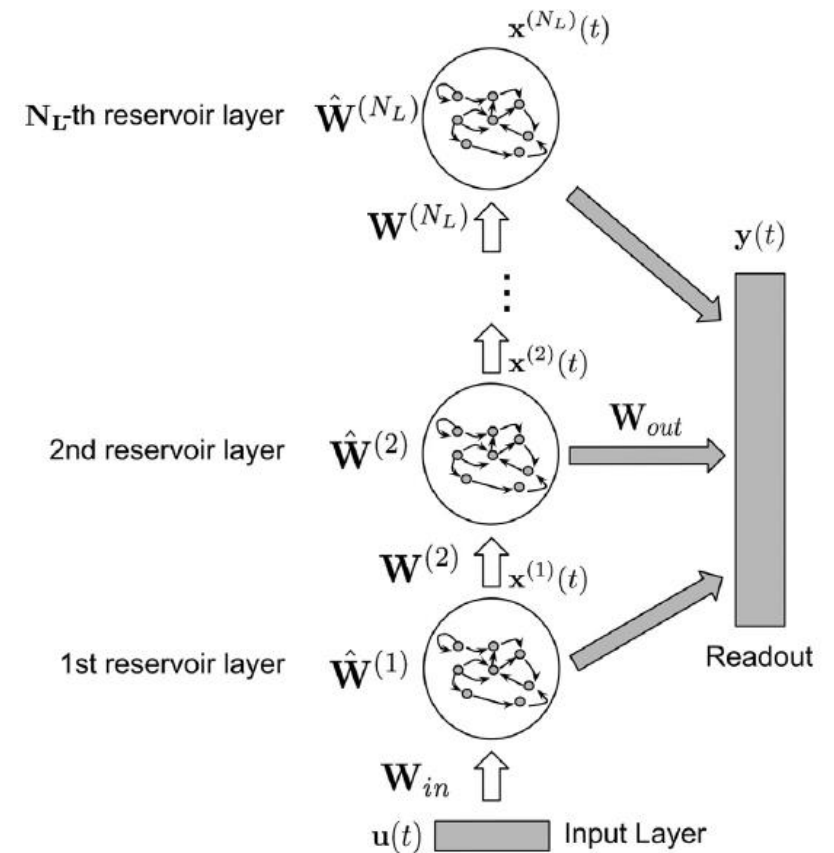


Deep ESNs

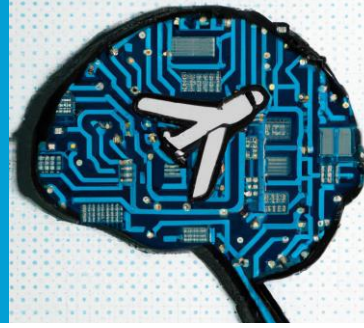
- Multiple-time-scale dynamics
- Richness of reservoir states: higher layers have less contractive dynamics
- Higher Memory Capacity:

$$MC_k = \frac{\text{cov}(x_{t-k}, y_t)^2}{\sigma(x_{t-k})^2 \sigma(y_t)^2}$$

where y_t is a prediction of input x_{t-k} .



Long/short-term memory (LSTM)

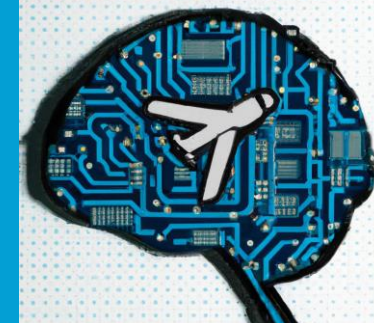


Devised to cope with the challenge of vanishing gradients

Introduces *gating* mechanisms to control the release of memory at a later instance in time.



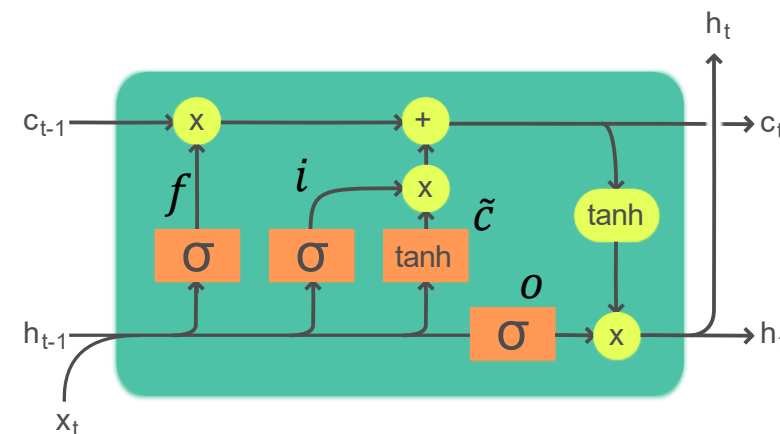
Long/short-term memory (LSTM)



Update of the LSTM:

$$\begin{aligned}f_t &= \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f) \\i_t &= \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i) \\o_t &= \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o) \\\tilde{c}_t &= g(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c) \\c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\h_t &= o_t \odot g(c_t)\end{aligned}$$

With σ the sigmoid function, and g the hyperbolic tangent function.



Legend:

Layer



Componentwise



Copy

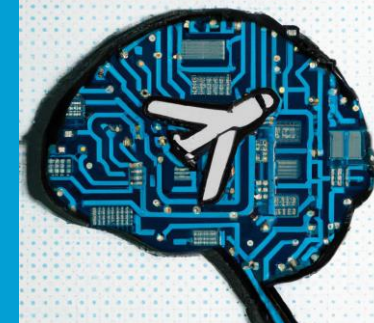


Concatenate



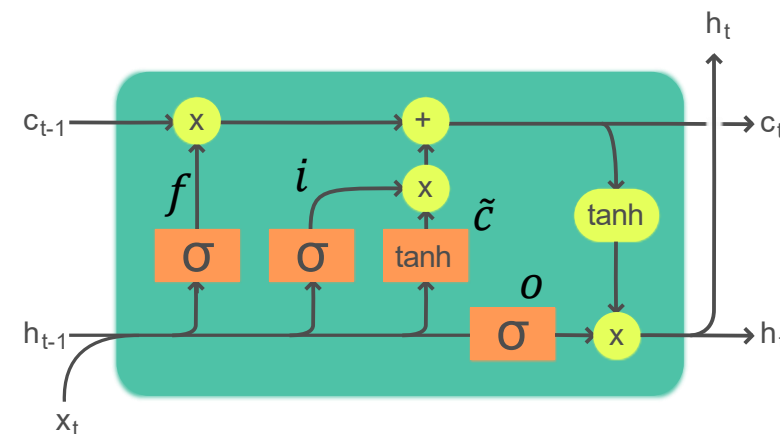
Guillaume Chevalier - Wikipedia

Long/short-term memory (LSTM)



Update of the LSTM:

$$\begin{aligned}f_t &= \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f) \\i_t &= \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i) \\o_t &= \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o) \\\tilde{c}_t &= g(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c) \\c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\h_t &= o_t \odot g(c_t)\end{aligned}$$



Legend:

Layer



Componentwise



Copy



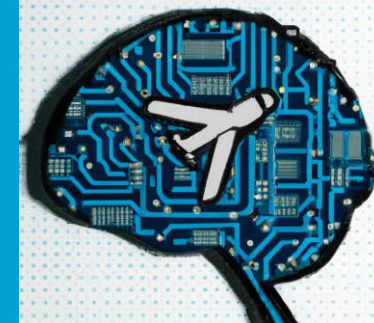
Concatenate



Guillaume Chevalier - Wikipedia

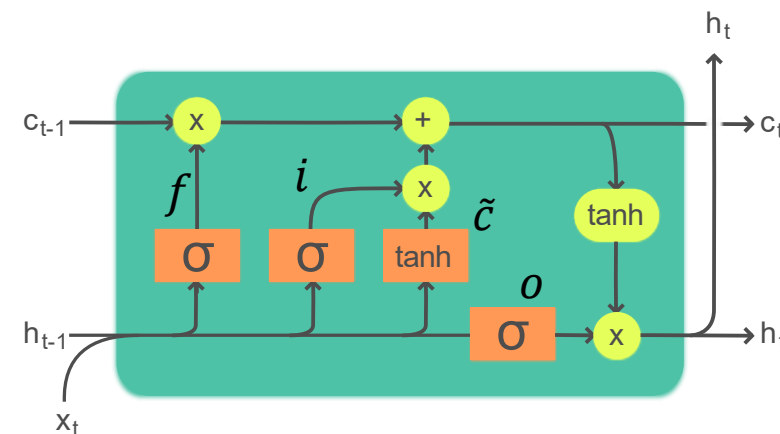
Forget gate: controls the decay of the cell state.
 $f_t = 1$ allows for a very long memory.

Long/short-term memory (LSTM)



Update of the LSTM:

$$\begin{aligned}f_t &= \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f) \\i_t &= \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i) \\o_t &= \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o) \\\tilde{c}_t &= g(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c) \\c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\h_t &= o_t \odot g(c_t)\end{aligned}$$



Legend:

Layer



Componentwise



Copy



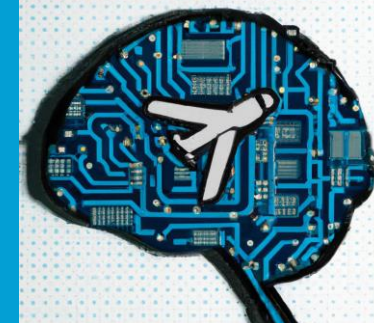
Concatenate



Guillaume Chevalier - Wikipedia

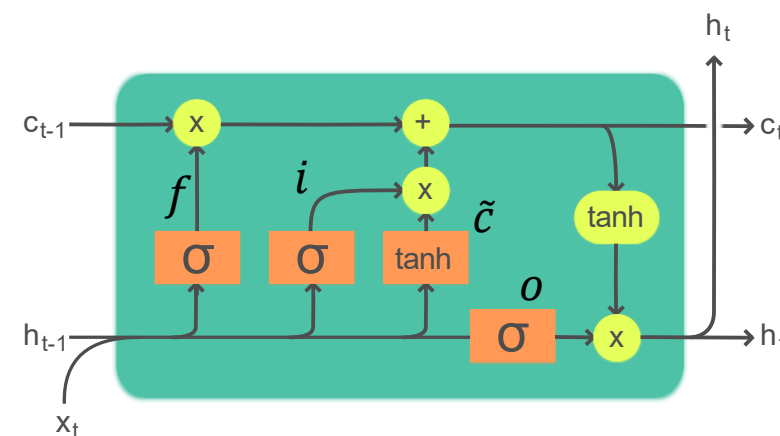
Input gate: whether to change the cell contents based on relevant inputs.

Long/short-term memory (LSTM)



Update of the LSTM:

$$\begin{aligned}f_t &= \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f) \\i_t &= \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i) \\o_t &= \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o) \\\tilde{c}_t &= g(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c) \\c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\h_t &= o_t \odot g(c_t)\end{aligned}$$



Legend:

Layer



Componentwise



Copy



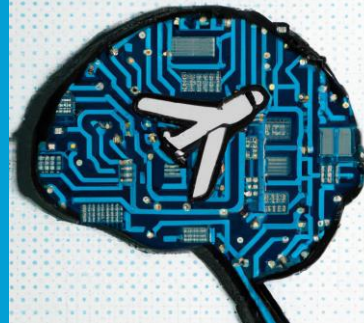
Concatenate



Guillaume Chevalier - Wikipedia

Output gate: Whether to pass on information to the outputs.

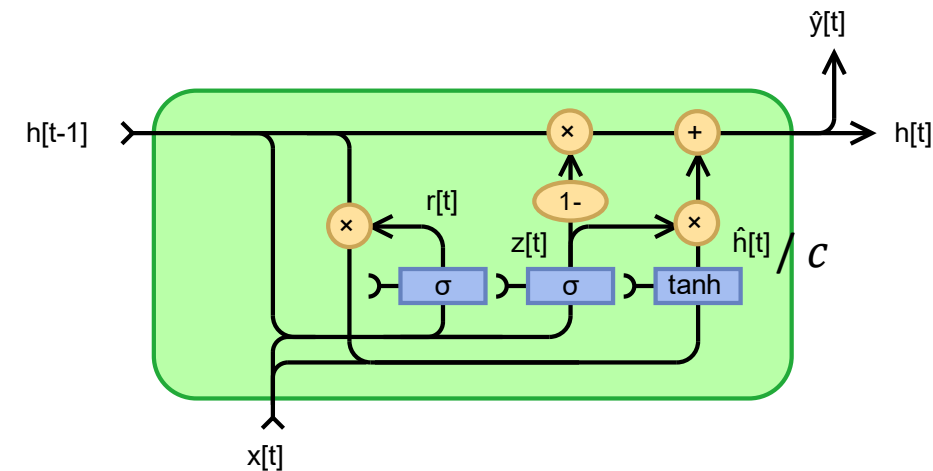
Gated Recurrent Unit (GRU)



Simplified version of the LSTM to obtain similar results with fewer parameters.

“Full” version:

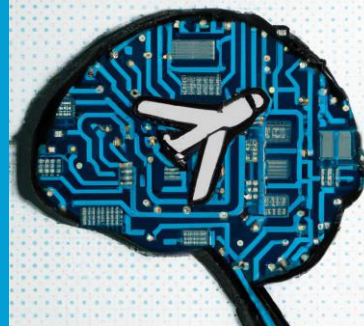
$$\begin{aligned}z_t &= \sigma(W_z \mathbf{x}_t + U_z \mathbf{h}_{t-1} + \mathbf{b}_z) \\r_t &= \sigma(W_r \mathbf{x}_t + U_r \mathbf{h}_{t-1} + \mathbf{b}_r) \\c_t &= g(W_c \mathbf{x}_t + U_c (r_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_c) \\h_t &= (1 - z_t) \odot \mathbf{h}_{t-1} + z_t \odot c_t\end{aligned}$$



Jeblad - Wikipedia

Where z_t is the “**update**” gate and r_t the “**reset**” gate.

Long/short-term memory (LSTM)



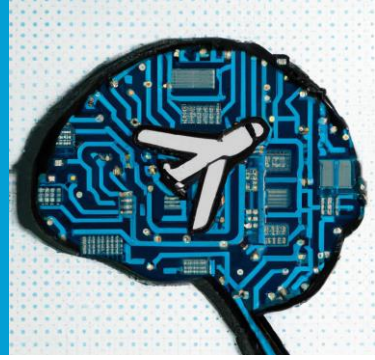
Advantages:

- ✓ Much improved long-term memory learning compared to vanilla recurrent neural networks.

Disadvantages:

- ✗ Additional parameters still make for quite a hard learning problem.
- ✗ Does not make use of modern hardware parallelization options.

Deep state space models

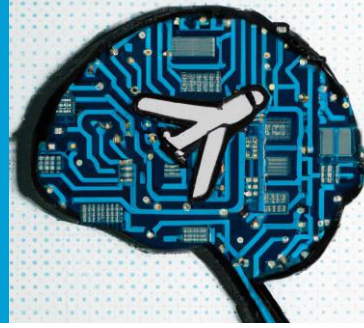


Combine the power of deep recurrent neural networks with the interpretability of state space models:

$$\mathbf{h}_t = A\mathbf{h}_{t-1} + B\mathbf{x}_t$$

$$y = C\mathbf{h}_t + D\mathbf{x}_t$$

Deep state space models



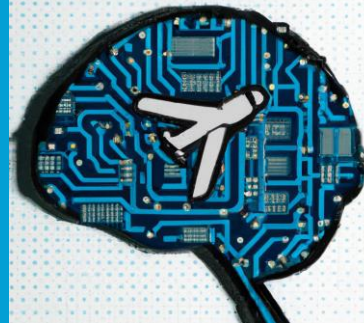
Combine the power of deep recurrent neural networks with the interpretability of state space models:

$$\mathbf{h}_t = A\mathbf{h}_{t-1} + B\mathbf{x}_t$$

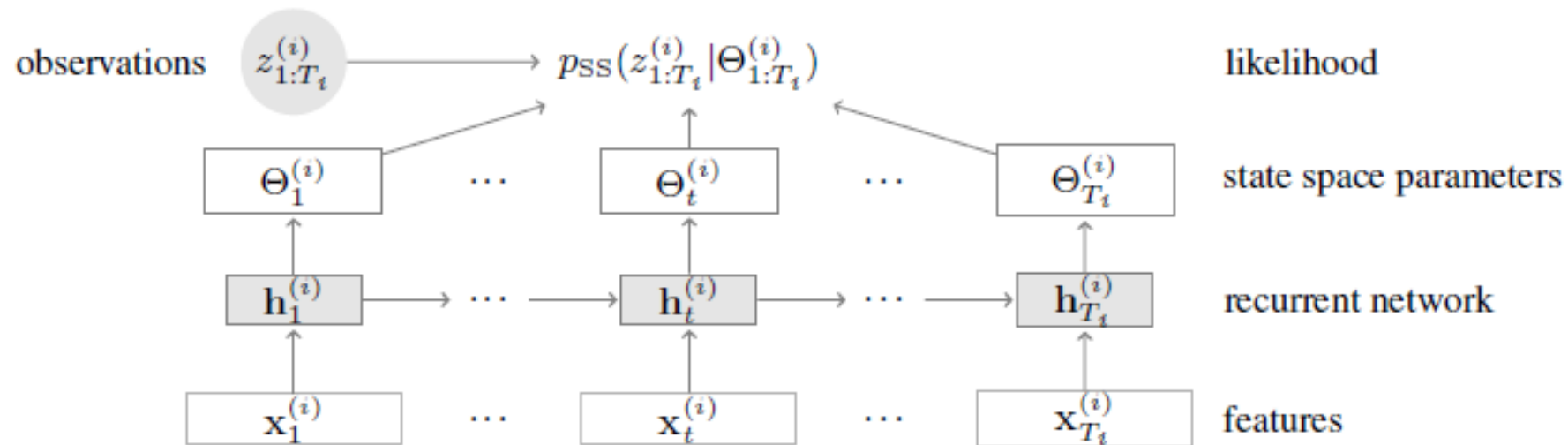
$$y = C\mathbf{h}_t + D\mathbf{x}_t$$

Does this look familiar? 😎

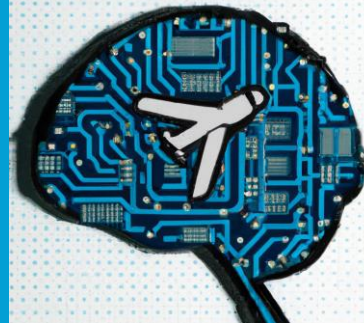
Deep SSMs



Combine the power of deep recurrent neural networks with the interpretability of state space models:



Structured State Space Sequence Models

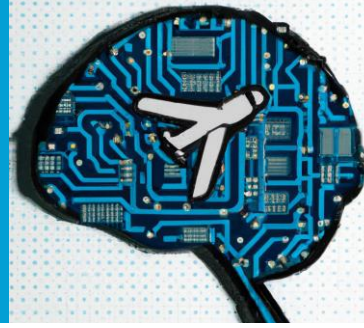


Parallellization

For a given part of the sequence, the SSM can be run by convolution.

$$\begin{aligned} \mathbf{h}_0 &= B\mathbf{x}_0, & \mathbf{h}_1 &= AB\mathbf{x}_0 + B\mathbf{x}_1, & \mathbf{h}_2 &= A^2B\mathbf{x}_0 + AB\mathbf{x}_1 + B\mathbf{x}_2, & \dots, \\ y_0 &= CB\mathbf{x}_0, & y_1 &= CAB\mathbf{x}_0 + CB\mathbf{x}_1, & y_2 &= CA^2B\mathbf{x}_0 + CAB\mathbf{x}_1 + CB\mathbf{x}_2, & \dots, \end{aligned}$$

Structured State Space Sequence Models



Parallelization

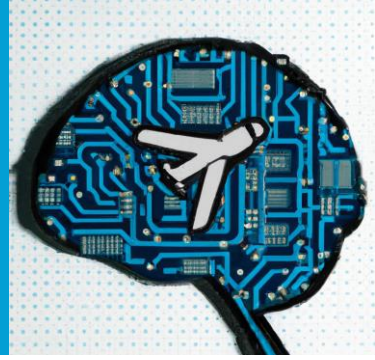
For a given part of the sequence, the SSM can be run by convolution.

$$\begin{aligned} \mathbf{h}_0 &= B\mathbf{x}_0, & \mathbf{h}_1 &= AB\mathbf{x}_0 + B\mathbf{x}_1, & \mathbf{h}_2 &= A^2B\mathbf{x}_0 + AB\mathbf{x}_1 + B\mathbf{x}_2, & \dots, \\ y_0 &= CB\mathbf{x}_0, & y_1 &= CAB\mathbf{x}_0 + CB\mathbf{x}_1, & y_2 &= CA^2B\mathbf{x}_0 + CAB\mathbf{x}_1 + CB\mathbf{x}_2, & \dots, \end{aligned}$$

Can be rewritten as:

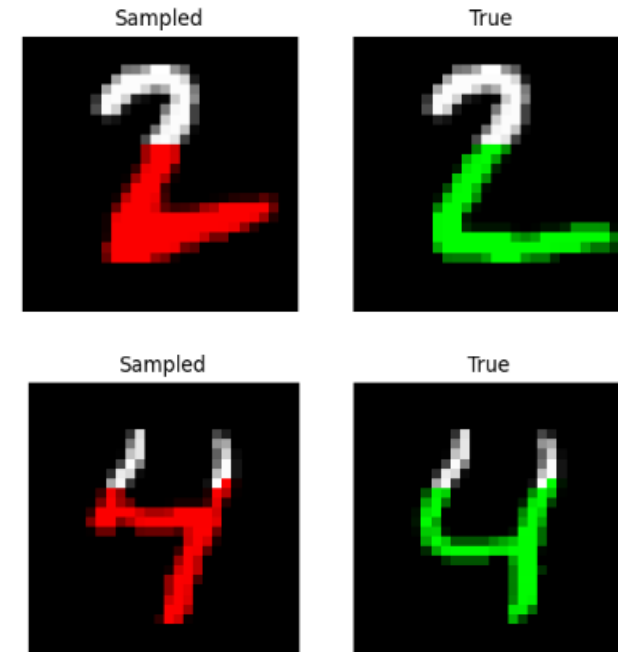
$$\begin{aligned} y &= K * \mathbf{x}_{1:t} \\ K &= (CA^tB, CA^{t-1}B, \dots, CB) \end{aligned}$$

Structured State Space Sequence Models



Structured state space sequence models (S4):

Imposing structure on A and employing the z-transform to allow for efficient, parallel computing of the outputs.

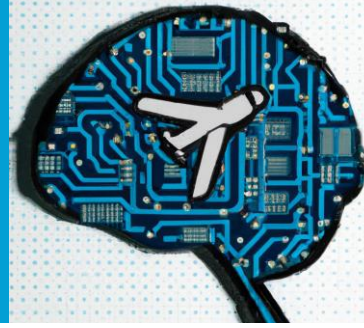


<https://iclr-blog-track.github.io/2022/03/25/annotated-s4/>

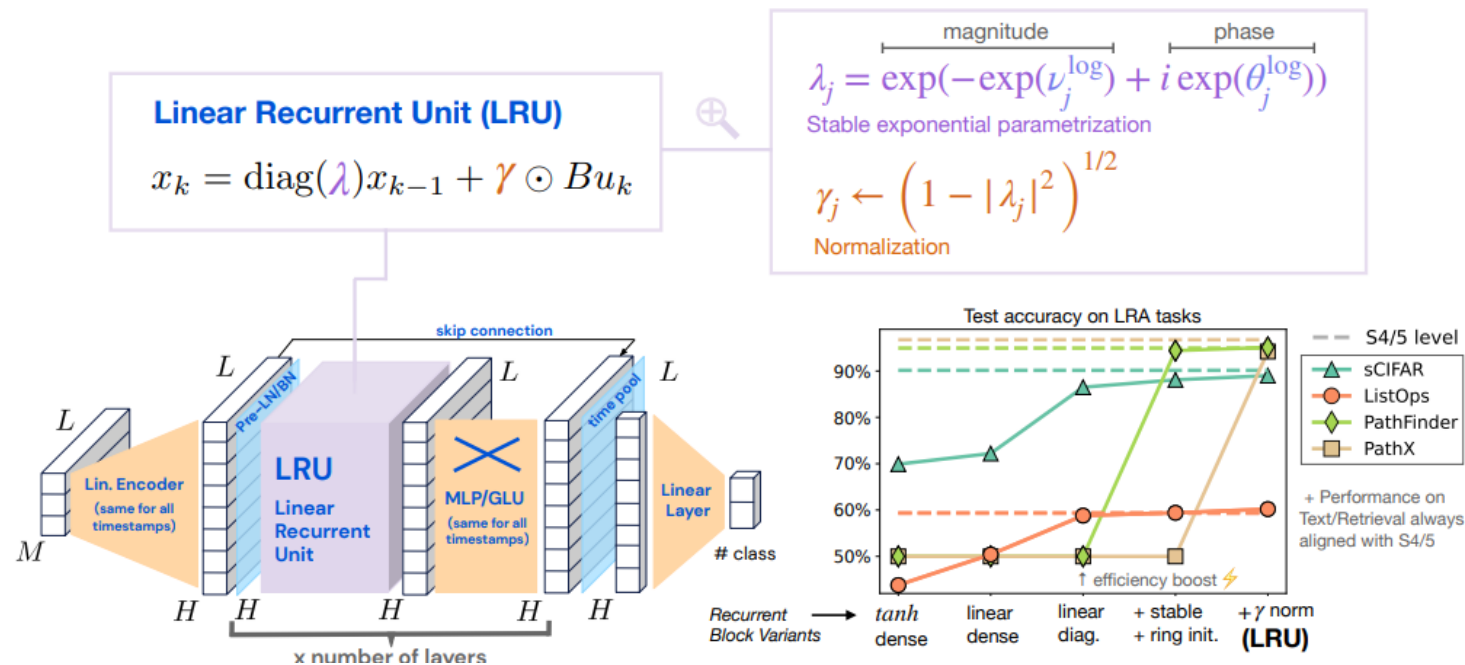
Gu, A., Dao, T., Ermon, S., Rudra, A., & Ré, C. (2020). Hippo: Recurrent memory with optimal polynomial projections. *Advances in neural information processing systems*, 33, 1474-1487.

Gu, A., Goel, K., & Ré, C. (2021). Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*.

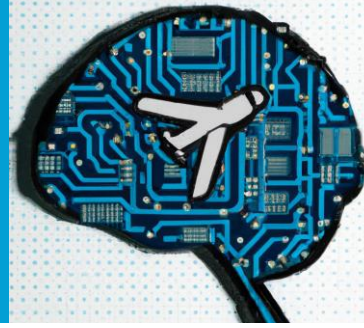
Linear Recurrent Units (LRUs)



Simpler version capturing main properties of deep SSMs.



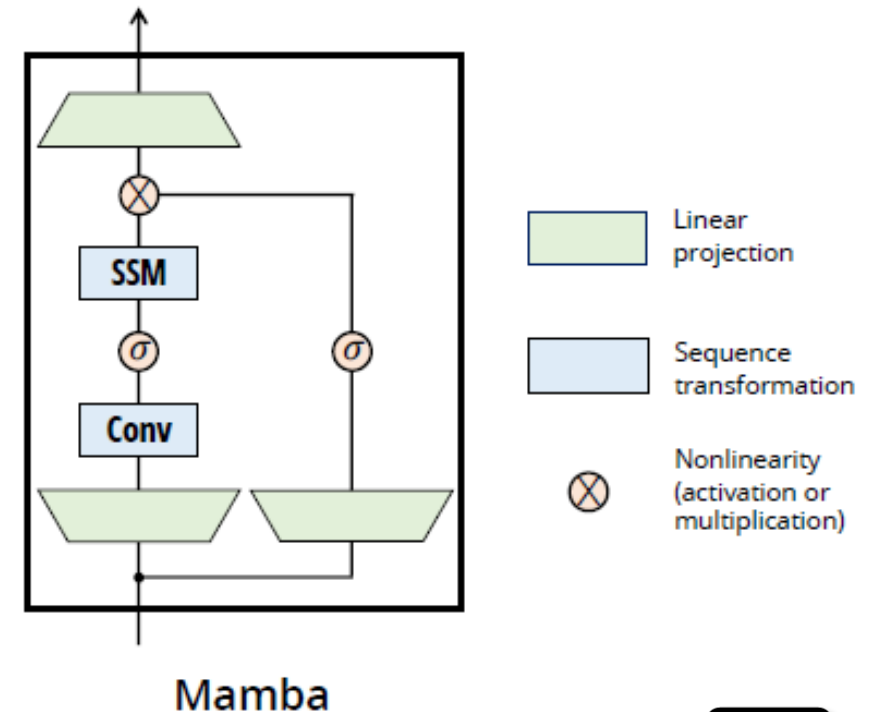
Dynamic LRUs



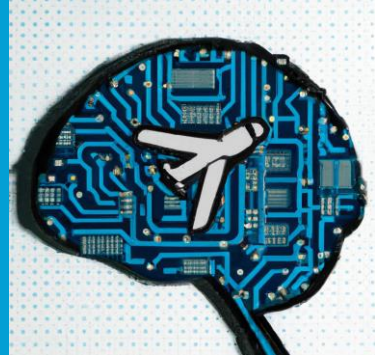
Mamba:

Make the parameters of the SSM dynamic over time, while keeping efficient computation.

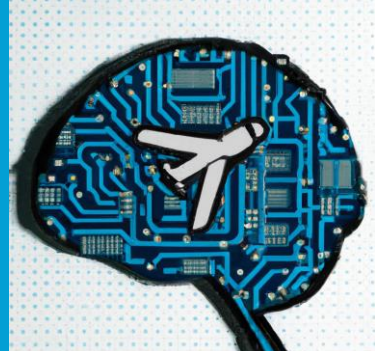
$$B, C, \Delta \rightarrow B(x_t), C(x_t), \Delta(x_t)$$



Conclusion

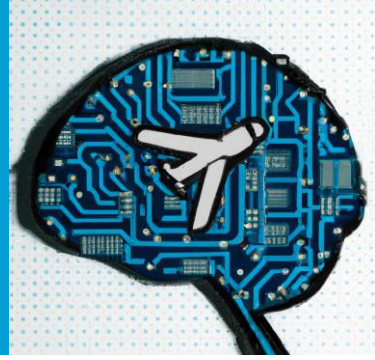


- Various recurrent neural networks exist that can successfully tackle sequential tasks
- Training these networks remains challenging, especially in terms of long-term dependencies or remembering detailed sequences over time



Quiz

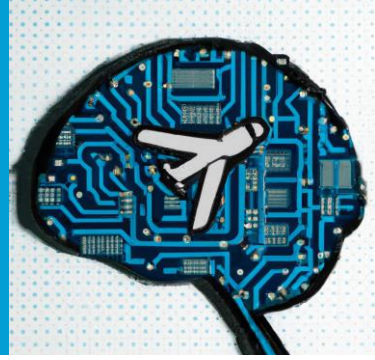
Question 1



What is the main problem of training a baseline recurrent neural network?

- A. It requires excessive memory to perform backpropagation as the sequence of inputs gets longer.
- B. The error gradients have a tendency to blow up or vanish when unrolling over time.
- C. When unrolling the network over time for backpropagation, the same weight has to be updated many times.
- D. It is not possible to attribute the error at time T to a specific input or hidden state at time $T - \Delta t$.

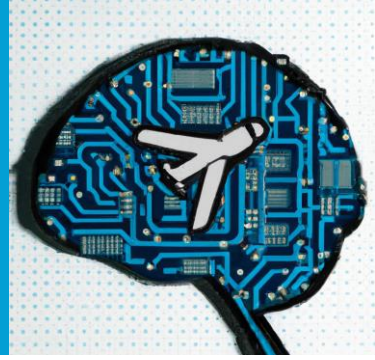
Question 1



What is the main problem of training a baseline recurrent neural network?

- A. It requires excessive memory to perform backpropagation as the sequence of inputs gets longer.
- B. The error gradients have a tendency to blow up or vanish when unrolling over time.
- C. When unrolling the network over time for backpropagation, the same weight has to be updated many times.
- D. It is not possible to attribute the error at time T to a specific input or hidden state at time $T - \Delta t$.

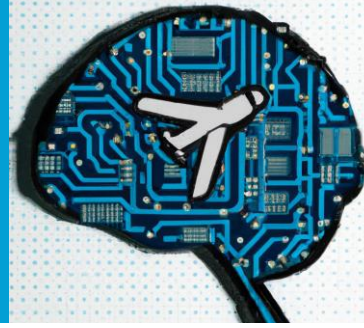
Question 2



What is the main reason that transformers can take input sequences of varying length?

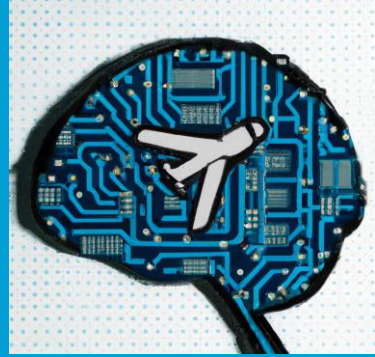
- A. When running the transformer, its internal state gets updated in an iterative fashion, which works for varying lengths.
- B. The weights of the transformer for different time steps are generated with a pre-determined function, applicable to any length.
- C. The transformer cannot take input sequences of varying length, as the weights are pre-determined.
- D. The weights for the subsequent inputs are generated based on the inputs themselves, making it applicable to varying input lengths.

Question 2



What is the main reason that transformers can take input sequences of varying length?

- A. When running the transformer, its internal state gets updated in an iterative fashion, which works for varying lengths.
- B. The weights of the transformer for different time steps are generated with a pre-determined function, applicable to any length.
- C. The transformer cannot take input sequences of varying length, as the weights are pre-determined.
- D. The weights for the subsequent inputs are generated based on the inputs themselves, making it applicable to varying input lengths.



Questions?